

Auto-Categorisation of ISIS Neutron and Muon Source Fault Logs

A report submitted to the University of Manchester for the degree of
Bachelor's of Science in the Faculty of Science and Engineering

Author: Sai Putravu
Student ID: 10829976
Supervisor: Professor Thomas Thomson
Word count: 14954

2025

School of Computer Science

Abstract

The ISIS Neutron and Muon Source is a large-scale research facility requiring complex machinery and sophisticated maintenance strategies to ensure operational availability for international scientific users. Predictive maintenance (PdM) policies offer potential for optimising maintenance and reducing costly downtime, but leveraging the facility’s text-based operational fault log (Operalog) is hindered by its unstructured nature. This project addresses the challenge of automatically categorising a subsection of log entries, the ion source failures, by developing and evaluating an unsupervised machine learning pipeline based on modern Natural Language Processing (NLP) techniques. The pipeline involves: (1) pre-processing of the **FaultDescription** field; (2) generating semantic vector representations using state-of-the-art sentence embedding models (MPNet and Nomic); (3) applying dimensionality reduction (UMAP); (4) performing unsupervised clustering (k-Medoids, DBSCAN, HDBSCAN); (5) optimising hyperparameters using Optuna with multiple clustering validity heuristics; and (6) automatically generating descriptive cluster labels using SpaCy.

The optimisation process identified HDBSCAN clustering on UMAP-reduced Nomic embeddings (favouring Euclidean distance and 5-6 components) with default text pre-processing as a particularly effective configuration, suggesting between 5 and 25 distinct fault categories within the ion source data.

Given the complexity and novelty of this task, this work focuses on demonstrating the feasibility of this approach and represents significant progress in developing tools for analysing specialised industrial logs. The primary contribution is the pipeline methodology itself. Future work is essential for refining the automatically generated labels to maximise their operational utility. This research paves the way for enhanced data-driven predictive maintenance at complex scientific facilities.

Acknowledgements

I would like to express my sincere gratitude to my supervisor, Professor Thomas Thomson, for his invaluable support throughout this project. His excellent advice and guidance were instrumental in shaping this work and motivating me during the research process.

I would also like to thank my family for their unwavering support during my university years.

Contents

List of Figures	v
List of Tables	vii
Abbreviations and Acronyms	viii
1 Introduction	1
1.1 Motivation	3
1.2 The ISIS Facility’s Complexity and Operational Challenges	4
1.2.1 Maintenance at ISIS	5
1.3 Maintenance Techniques	5
1.4 Data	8
1.4.1 The Operalog	9
2 Natural Language Processing Background	12
2.1 Sentence Similarity	12
2.2 Sentence Embedding	13
2.2.1 Transformers and Foundational Pre-trained Language Models	14
2.3 Dimensionality Reduction	16
2.3.1 PCA	16
2.3.2 UMAP	17
2.4 Clustering	18
2.4.1 k-Medoids	19
2.4.2 DBSCAN	19
2.4.3 HDBSCAN	21
3 Automatic Categorisation and Label Generation	23
3.1 Overview	23
3.2 Natural Language Pre-processing	24
3.3 Text Embedding Model Selection and Application	25
3.3.1 Dataset and Hardware Driven Requirements	25
3.3.2 Candidate Models	26
3.3.3 Section Summary	28
3.4 Embedding Dimensionality Reduction	28

3.4.1	Visualisation	29
3.4.2	UMAP Hyperparameters	30
3.4.3	Section Summary	32
3.5	Unsupervised Categorisation Through Clustering	32
3.5.1	Clustering Candidates	33
3.5.2	Clustering heuristics	34
3.6	Hyperparameter optimisation	35
3.7	Label generation	37
3.8	CLI Application	38
4	Results and Discussion	39
4.1	Experiments and Results	40
4.1.1	Head-to-Head (H2H) Phase	40
4.1.2	Free-For-All (F4A) Phase	41
4.2	Distribution of Hyperparameters	42
4.3	Discussion	43
5	Conclusion	46
	Bibliography	49
A	Results and Figures	56
A.1	UMAP Hyperparameter Search	56
A.2	Experimental Results Plots	60
A.2.1	Aggregated Configuration Distributions	60
A.2.2	H2H Optimal Configuration Plots	64
A.2.3	H2H Cluster Size Distributions	66
A.2.4	F4A Optimal Configuration Plots	70
A.2.5	F4A Cluster Size Distributions	72
B	Supplementary Technical Details	76
B.1	BERT and XLNet Pre-training	76
B.1.1	BERT Pre-training Tasks	76
B.1.2	XLNet Technical Details	76
B.2	MPNet Mathematical Formulations	78
B.2.1	MPNet Pre-training Objective	78
B.2.2	MPNet Input Structuring Example	79
B.2.3	PCA Formalism and SVD Derivation	79
B.3	k-Medoids - The PAM Algorithm	81

List of Figures

1.1	The schematic representation of the physical layout of ISIS. The light grey areas are footprints of the buildings. Source: (Thomason, 2019).	6
1.2	ISIS ion source and chain of accelerators, with H^- ions in blue and protons in red. Not to scale. Source: (ISIS Neutron and Muon Source, 2021).	7
1.3	Active versus Downtime days (2016 - 2024), according to the ISIS operational cycle. Data source: (ISIS Neutron and Muon Source, 2024)	7
1.4	Operalog Field Length Distributions	10
1.5	Comparison of word clouds for the top 300 most common words for the Fault-Repair field versus the FaultDescription field. Larger words are more common and collocations are enabled so both words and phrases are shown, leading to some repeating.	11
2.1	Example text embedding with Nomic Text Embedding v1.5 [62] of random sentences generated using OpenAI's ChatGPT o3-mini-high and projected onto 2 dimensions using UMAP [53] with a random seed of 42 and minimum neighbours of 15. Only some of the labels are highlighted for visual clarity.	14
2.2	An illustration of UMAP applied (right) on samples taken from the skeletal structure of an elephant in 3-dimensions (left) with the UMAP hyperparameter values displayed. Highlighted, a black sphere (left) and circle (right), shows a chosen point mapped from 3-dimensions to 2. Source: (Coenen, Andy and Pearce, Adam, 2020).	18
3.1	An overview of the end-to-end machine learning pipeline, outlining each key stage and highlighting which sections required hyperparameter optimisation.	24
3.2	Histogram plot of 100 UMAP runs' normalised variance (Equation 3.1) is shown for both MPNet and Nomic embedding spaces of the Operalog FaultDescription entries. UMAP parameters: <code>min_dist = 0.1</code> , <code>n_neighbors = 15</code> , <code>metric = cosine</code> .	30
3.3	Comparison of PCA versus UMAP dimensionality reduction on the MPNet (top row) and Nomic (bottom row) embedding spaces on Operalog FaultDescription entries Normalised variance (Equation 3.1) is shown. UMAP parameters: <code>min_dist = 0.1</code> , <code>n_neighbors = 15</code> , <code>metric = cosine</code> .	31
3.4	IsoScore comparison of MPNet versus Nomic embeddings over 100 runs of UMAP. UMAP parameters: <code>min_dist = 0.1</code> , <code>n_neighbors = 15</code> , <code>metric = cosine</code> .	34

A.1	MPNet, grid search for UMAP hyperparameter (see Table 3.4) over the parameters <code>metric</code> and <code>min_dist</code>	57
A.2	MPNet, grid search for UMAP hyperparameter (see Table 3.4) over the parameters <code>min_dist</code> and <code>n_neighbors</code>	57
A.3	MPNet, grid search for UMAP hyperparameter (see Table 3.4) over the parameters <code>metric</code> and <code>n_neighbors</code>	58
A.4	Nomic, grid search for UMAP hyperparameter (see Table 3.4) over the parameters <code>metric</code> and <code>min_dist</code>	58
A.5	Nomic, grid search for UMAP hyperparameter (see Table 3.4) over the parameters <code>min_dist</code> and <code>n_neighbors</code>	59
A.6	Nomic, grid search for UMAP hyperparameter (see Table 3.4) over the parameters <code>metric</code> and <code>n_neighbors</code>	59
A.7	Overall distribution of hyperparameter values across all optimal configurations identified by Optuna in H2H and F4A studies (top), and for the top 5 configurations selected via rank aggregation from each study (bottom).	61
A.8	Distribution of hyperparameter values for MPNet embeddings across all optimal configurations identified by Optuna in H2H and F4A studies (top), and for the top 5 configurations selected via rank aggregation from each study (bottom).	62
A.9	Distribution of hyperparameter values for MPNet embeddings across all optimal configurations identified by Optuna in H2H and F4A studies (top), and for the top 5 configurations selected via rank aggregation from each study (bottom).	63
A.10	Heuristic trade-offs for optimal configurations from H2H Optuna runs using MPNet embeddings. Best solutions highlighted in red.	64
A.11	Heuristic trade-offs for optimal configurations from H2H Optuna runs using Nomic embeddings. Best solutions highlighted in red.	65
A.12	Cluster size distributions over 10 runs for optimal H2H configurations using MPNet embeddings. Empty plots indicate no optimal configurations were found for that method.	67
A.13	Cluster size distributions over 10 runs for optimal H2H configurations using Nomic embeddings. Empty plots indicate no optimal configurations were found for that method.	69
A.14	Heuristic trade-offs for optimal configurations from F4A Optuna runs using MPNet embeddings, varying pre-processing flags. Best solutions highlighted in red.	70
A.15	Heuristic trade-offs for optimal configurations from F4A Optuna runs using Nomic embeddings, varying pre-processing flags. Best solutions highlighted in red.	71
A.16	Cluster size distributions over 10 runs for optimal F4A configurations using MPNet embeddings, varying pre-processing flags.	73
A.17	Cluster size distributions over 10 runs for optimal F4A configurations using Nomic embeddings, varying pre-processing flags.	75
B.1	BERT Pre-training and Fine-tuning Phases (Appendix)	77

List of Tables

1.1	Examples of applications of PdM for industrial maintenance strategies.	9
1.2	Description of important features (columns) in the Operalog.	10
3.1	Operational crew abbreviation mapping.	25
3.2	Research hardware description.	26
3.3	Models considered but excluded from the embedding pipeline.	27
3.4	UMAP hyperparameter values explored in naive grid search	32
3.5	Optuna hyperparameter search space. k-Medoids hyperparameter <i>k</i> was re-named to <code>n_clusters</code> ; DBSCAN hyperparameter <code>MinPts</code> was renamed to <code>min_cluster_size</code> ; and HDBSCAN hyperparameter <i>m_{pts}</i> was renamed to <code>min_pts</code>	36
4.1	Summary of H2H optimisation runs (default pre-processing). All H2H runs used default pre-processing settings (<code>ignore_case=True</code> , <code>keep_punctuation=False</code>). Appendix references point to Appendix A.2.	40
4.2	Summary of Free-For-All (F4A) Optimisation Runs. Appendix references point to Appendix A.2. Pre-processor settings <code>ignore_case</code> (normalise text casing) and <code>keep_punc</code> (keep punctuation) are highlighted. See Section 3.2 for more information.	41
4.3	Summary of Optuna optimal hyperparameter plots. The top 5 hyperparameter plots are selected via rank aggregation. Appendix references point to Appendix A.2. Parameters correspond to Table 3.5 See Section 3.6 for more information.	42

Abbreviations and Acronyms

AE Auto-encoding.	IoT Internet of Things.
AR Auto-regressive.	LEBT Low energy beam transport line.
BERT Bidirectional Encoder Representation from Transformers.	LINAC Linear Accelerator.
CBM Condition-based maintenance policy.	LSA Latent Semantic Analysis.
CLI Command Line Interface.	MCR Main Control Room.
CNN Convolutional Neural Network.	ML Machine Learning.
CPU Central Processing Unit.	MLM Masked Language Modelling.
CUDA Compute Unified Device Architecture.	MRL Matryoshka Representation Learning.
DBSCAN Density-Based Spatial Clustering of Application with Noise.	MST Minimum Spanning Tree.
DL Deep Learning.	MTEB Massive Text Embedding Benchmark.
ELMo Embeddings from Language Models.	NLP Natural Language Processing.
EPB1 Extracted proton beam line 1.	Operalog The ISIS Operational Log.
EPB2 Extracted proton beam line 2.	PAM Partitioning Around Medoids.
F4A Free-For-All.	PCA Principle Component Analysis.
FAP Fault Analysis Pathway.	PdM Predictive maintenance policy.
FLD First-line diagnosis system.	PLM Pre-trained Language Model.
GPT Generative Pre-trained Transformer.	PvM Preventative maintenance policy.
GPU Graphics Processing Unit.	R2F Run-to-failure maintenance policy.
H2H Head-to-Head.	RAM Random Access Memory.
HDBSCAN Hierarchical Density-Based Spatial Clustering of Application with Noise.	RFQ Radio-frequency quadrupole.
HDD Hard-Disk Drive.	RF Random Forest.
HEDS High energy drift space.	RoPE Rotary Positional Embeddings.
	SAFE Supervised Aggregative Feature Extraction.
	SMART Self-monitoring and reporting technology.

STFC	Science and Technology Facilities Council.	t-SNE	t-distributed Stochastic Neighbour Embedding.
SVD	Singular Value Decomposition.	UKRI	United Kingdom Research and Innovation.
SVM	Support Vector Machine.	UMAP	Uniform Manifold Approximation and Projection.
TS-1	Target Station 1.	VRAM	Video Random Access Memory.
TS-2	Target Station 2.		

Chapter 1

Introduction

The Science and Technology Facilities Council’s (STFC) ISIS Neutron and Muon Source, located at the Rutherford Appleton Laboratory, is a research centre enabling studies across physical and life sciences by producing beams of neutrons and muons [88]. Generating these beams requires large, intricate accelerator systems operating under extreme and demanding conditions [34]. The inherent complexity means component failures are inevitable, making effective maintenance crucial for minimising downtime and maximising scientific output [88].

While the ISIS Neutron and Muon Source (hereafter, referred to as ISIS for brevity) employs various maintenance strategies, leveraging predictive maintenance (PdM) techniques holds significant promise [88, 85]. PdM aims to predict failures based on data analysis, enabling proactive intervention [18]. The facility’s Operational Log (Operalog) contains decades of valuable text entries detailing faults and their remediation. However, its unstructured nature, domain-specific terminology, and variability currently hinder systematic analysis for PdM [90].

This project tackles this challenge by exploring the application of modern Natural Language Processing (NLP) and unsupervised machine learning techniques to automatically categorise these textual entries. It is crucial to recognise the scale and open-ended nature of this research endeavour - there is no established ‘industry standard’ for automatically categorising such specialised industrial operational logs [18]. Therefore, this project’s primary aim is not to deliver a final, production-ready system, but rather to progress the state-of-the-art by developing and rigorously evaluating an innovative pipeline for this task. The core contribution lies in the methodology itself - through exploring the feasibility, comparing different technical approaches, performing detailed parameter analysis, and building a tool that demonstrates a pathway towards automated insight extraction from these complex logs. This work constitutes one part of a two-part collaborative project; the partner project focuses on analysing structured data from specialised ion source logs.

The specific objectives of this project, framed within this exploratory context, are:

1. To pre-process and clean the unstructured text data from the Operalog (the `FaultDescription` field) with a focus on ‘ion source’ entries.
2. To investigate and compare the effectiveness of different state-of-the-art sentence em-

bedding models (MPNet [79], Nomic [62]).

3. To evaluate dimensionality reduction techniques (PCA [67] vs. UMAP [53]).
4. To apply and compare different unsupervised clustering algorithms (k-Medoids [38], DBSCAN [26], HDBSCAN [16]) to identify potential fault categories.
5. To implement a systematic hyperparameter optimisation strategy, exploring variations in dimensionality reduction and clustering parameters using Optuna [4], based on internal clustering evaluation metrics [72, 23, 15] to identify promising pipeline configurations.
6. To develop and demonstrate a method for automatically generating initial descriptive labels for the discovered clusters.

The evaluation strategy involves assessing the technical success of the pipeline components and the relative performance of different configurations. As ground truth fault categories are unavailable, the evaluation relies on internal clustering validation metrics, a standard approach in unsupervised learning. Specifically, the Silhouette coefficient, Davies-Bouldin index, and Calinski-Harabasz index metrics were chosen based on their widespread use and ability to capture different aspects of cluster cohesion and separation [72, 23, 15]. To systematically identify optimal pipeline parameters under these multiple objectives, the Optuna framework [4] was utilised, leveraging its capabilities for efficient multi-objective search. This involved a detailed exploration of how choices in embedding models, distance metrics, dimensionality reduction settings, and clustering algorithm parameters influenced the final clustering quality based on these heuristics. Alongside these quantitative indicators, qualitative assessment will consider the coherence of the discovered clusters with domain knowledge of the Operalog and of the ISIS facility.

This report is structured as follows:

- **Chapter 2 (Natural Language Processing Background)** - Provides the necessary theoretical background on the core NLP and machine learning techniques employed.
- **Chapter 3 (Automatic Categorisation and Label Generation)** - Details the developed methodology, data handling, specific tool choices, hyperparameter optimisation framework, and the label generation approach.
- **Chapter 4 (Results and Discussion)** - Presents experimental results demonstrating the pipeline’s capability to automatically structure the log entries and compares the performance of different embedding, dimensionality reduction, and clustering choices. This chapter additionally discusses the implications of these findings, including the hyperparameter optimisation outcomes, particularly favouring the HDBSCAN algorithm [16] under specific parametrisations for this dataset, and reflects on the overall feasibility of the approach.
- **Chapter 5 (Conclusion)** - Summarises the project’s progress, highlighting the successful development and evaluation of the auto-categorisation pipeline, reflects on limitations, and outlines crucial next steps for future research, particularly concerning label refinement and validation.

1.1 Motivation

The primary motivation for this research is the potential to gain insight on information within the ISIS Operalog’s extensive unstructured text data. Manual analysis is labour-intensive and potentially misses subtle patterns which are crucial for effective predictive maintenance (PdM) [90, 12, 85, 84]. Automating the categorisation of fault reports promises to reduce this burden, enable faster diagnostics, and facilitate the identification of recurring failure modes, directly supporting data-driven PdM strategies [61, 3, 18].

However, the specific characteristics of the Operalog - its unstructured format, domain-specific language, semantic nuance, and variable entry length - make this auto-categorisation task particularly challenging. Simple methods are insufficient, necessitating the exploration of advanced NLP techniques capable of understanding semantic meaning. This project is therefore motivated by the opportunity to significantly advance the methodology for analysing such specialised operational logs. By constructing and evaluating a novel pipeline incorporating state-of-the-art sentence embeddings [79, 62] and unsupervised clustering [38, 26, 16], we aim to demonstrate the potential of these techniques and provide a foundational tool for future, more refined systems. The progress achieved in developing and evaluating this pipeline constitutes the core technological advancement delivered by this work.

The specific research aims stemming from this motivation are:

- To determine if modern NLP techniques can effectively group semantically similar fault descriptions from the unstructured Operalog text.
- To compare the performance of different embedding models, dimensionality reduction techniques, and clustering algorithms within this specific application context.
- To develop a methodology for systematically optimising the parameters of this unsupervised pipeline to achieve the most coherent and meaningful categorisation possible within this exploratory phase.
- To explore the possibility of automatically generating human-interpretable labels for the discovered fault categories, acknowledging this as an initial step requiring further development.
- The input data is the pre-processed `FaultDescription` text which is scoped to ‘ion source’ during this project, with the hope to extend the pipeline more generally in future work.

The expected output is a version of this dataset augmented with cluster IDs and preliminary generated labels, alongside a comparative evaluation of the pipeline configurations explored. The evaluation approach, detailed in Chapter 4, relies on internal clustering heuristics (Silhouette coefficient, Davies-Bouldin index, Calinski-Harabasz index) to guide hyperparameter optimisation and assess cluster quality quantitatively [72, 23, 15]. Successfully demonstrating the pipeline’s ability to structure this complex data represents significant progress towards leveraging the Operalog for improved maintenance and operational reliability at ISIS.

1.2 The ISIS Facility’s Complexity and Operational Challenges

The ISIS Neutron and Muon Source at STFC Rutherford Appleton Laboratory is a large-scale research facility enabling physical and life sciences study through the production of neutron and muon beams [88]. Achieving this requires a complex, multi-stage accelerator system, the operation and maintenance of which present significant challenges relevant to this project.

The core process, described in *A Practical Guide to the ISIS Neutron and Muon Source* [34], involves accelerating negative hydrogen (H^-) ions through four major stages: an ion source, a radio-frequency quadrupole (RFQ), a linear accelerator (LINAC), and finally a large synchrotron ring. This chain ultimately produces an 800 MeV proton beam directed towards one of two target stations (TS-1 or TS-2) for neutron or muon generation [34]. A schematic overview is shown in Figure 1.1, with the particle path illustrated in Figure 1.2.

Successful operation across these stages demands extreme precision and stability. Throughout the entire acceleration process, maintaining an ultra-high vacuum (between 10^{-8} and 10^{-9} atmospheric pressure) is crucial, requiring tens of vacuum pumps and continuous monitoring, as leaks can disrupt the beam and damage sensitive components [34]. The initial H^- ion source must provide a stable, consistent beam, as inconsistencies can propagate and amplify downstream; component wear within the source itself is a common maintenance task requiring immediate intervention [34]. Accelerator structures, like those in the LINAC which utilise high-Q radio-frequency fields [54], require exceptional stability achieved through precise temperature regulation (significantly under $1^\circ C$ variation) to mitigate thermal effects of expansion and contraction [34].

Within the 163m circumference synchrotron, magnetic fields and accelerating voltages must be ramped in precise synchronism over nearly 8000 revolutions to boost the beam energy to 800 MeV. Any deviation risks beam oscillation, particle loss, and component damage which necessitates continuous, precise monitoring of beam position and machine parameters via sensor networks [34]. Even the injection mechanism, which uses charge-exchange via a thin foil to convert H^- ions to protons [34, 6], relies on the integrity of this consumable foil, whose replacement is a scheduled maintenance activity and whose damage causes immediate performance loss [34].

The complexity extends to beam transport and target stations. Extracted proton beams must be accurately steered by magnets towards the target stations; mis-steering can damage equipment, hence the need for heavy shielding and beam loss monitors [34]. At the target stations, the high-energy beam strikes targets to produce neutrons or muons [77, 34]. These target systems operate under extreme conditions and require careful monitoring and management to prevent overheating or material damage, representing another major operational and maintenance concern [34].

This overview highlights the key takeaway: the ISIS facility involves an intricate sequence of precisely controlled stages, creating numerous potential points of failure. While precautions like beam loss monitors and physical shielding exist, the inherent complexity and harsh operating environment (radiation, high-power) mean that component degradation and operational faults are inevitable. Effectively managing the facility therefore necessitates robust maintenance strategies and sophisticated diagnostic capabilities. Documenting fault events,

diagnoses, and repairs in operational logs (like the Operalog studied here) is essential, and the technical complexity of the facility directly influences the nature and variety of entries found in these logs, motivating the need for advanced analysis techniques like those explored in this report.

1.2.1 Maintenance at ISIS

As detailed in the 33-year historical account of the ISIS facility [88], the ISIS operations occur in cycles, periods of roughly 30-50 days where the machine runs constantly without breaks. Gaps between cycles typically range from 1 week to 3 months. In addition, typically every four years, shutdowns scheduled for 6-9 months occur for major maintenance and upgrade work. On the ground, day-to-day operations are run from the Main Control Room (MCR) by the ISIS crew which consists of 6 shift teams of the following roles: (1) duty officer, (2) assistant duty officer and (3) duty technician. The expertise and rapid response capabilities of these trained operational crews are vital for both routine operation and initial fault response. Minimising this downtime is crucial for several reasons inherent to operating a large-scale user facility like ISIS [88]. Primarily, unscheduled downtime directly impacts the international researchers allocated beam time, potentially jeopardising experiments planned months or years in advance and wasting valuable research opportunities. Furthermore, interruptions disrupt the tightly packed operational schedule, reduce the overall scientific output, and incur significant operational costs without delivering the facility’s core research product [88]. Therefore, understanding the factors that contribute to downtime and implementing strategies to mitigate it are paramount. Many factors affect downtime such as having a robust plan, the day-to-day operators’ knowledge of the system and adequate inventories of spares [88]. Figure 1.3 shows the ratio of downtime to active machine operation since 2016, indicating this machine has been down for maintenance over half the time since 2016. In other words, over half the time, the machine is not active and is undergoing maintenance. This highlights the trade-off between maximising operational availability and minimizing failures through planned maintenance. The major maintenance strategies and their trade-offs are further explored in Section 1.3.

To help reduce downtime, over the last few years, a first-line diagnosis (FLD) system was introduced, presented in (Asim Yaqoob, 2017). The FLD system helps reduce downtime by providing expert guidance on fault diagnosis and resolution, which has been shown to improve the dissemination of knowledge from experts to operations. The FLD utilises Fault Analysis Pathways (FAPs), which provide structural links between ISIS subsystems. This allows users of the system to access granular subsystems’ local documentation minimising file hunting and saving time and effort.

1.3 Maintenance Techniques

In industry, production system uptime is strongly coupled with equipment maintenance. Maintenance, once seen as a ‘necessary evil’, is now often viewed as a ‘profit contributor’ for maintaining a competitive edge [93, 27]. For research facilities like ISIS, maintenance directly impacts downtime and operational costs. Consequently, to minimise unexpected downtime

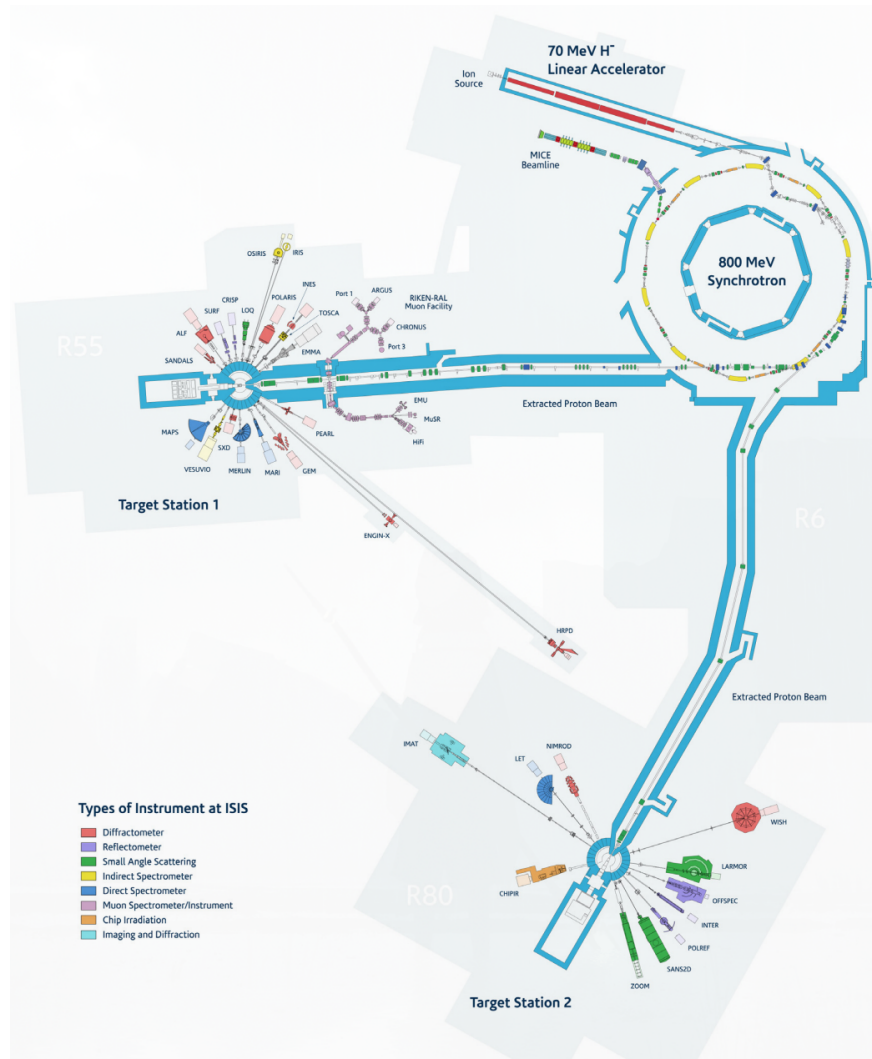


Figure 1.1: The schematic representation of the physical layout of ISIS. The light grey areas are footprints of the buildings. Source: (Thomason, 2019).

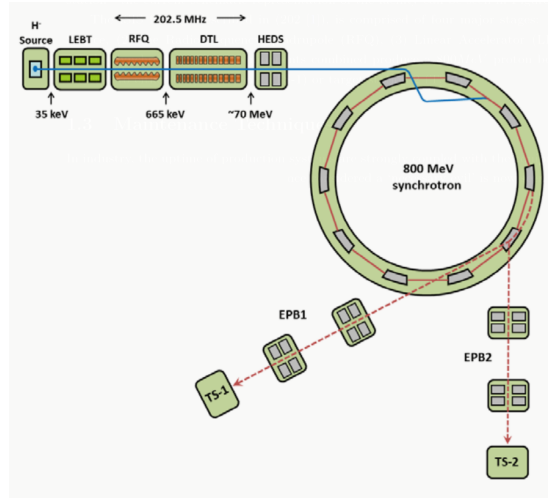


Figure 1.2: ISIS ion source and chain of accelerators, with H^- ions in blue and protons in red. Not to scale. Source: (ISIS Neutron and Muon Source, 2021).

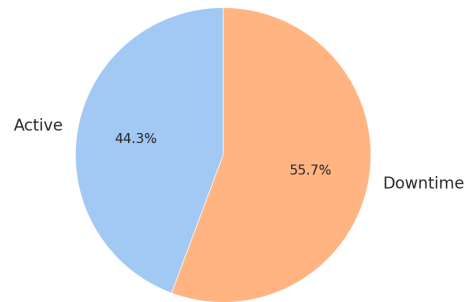


Figure 1.3: Active versus Downtime days (2016 - 2024), according to the ISIS operational cycle. Data source: (ISIS Neutron and Muon Source, 2024)

and maximise efficiency, many industrial applications collect vast quantities of lifecycle data, potentially including process information, events, and alarms from various system components [18].

In the literature, various terms and categories of maintenance arise each with differing strategies [85, 57, 84]. Thus, while there exists some disagreement in nomenclature, we consider the four categories presented in (Susto et al., 2012). The four maintenance policy categories are as follows, noting that each policy has unique benefits and drawbacks:

1. **Run-to-failure (R2F) maintenance** - Operate equipment until failure, then restore. This simple approach is typically costly due to unplanned downtime and potential need for complete component replacement.
2. **Preventative maintenance (PvM)** - Scheduled maintenance performed at regular intervals to increase longevity or anticipate end-of-life. While preventing some failures, it can waste resources on healthy components, causing unnecessary downtime.
3. **Condition-based maintenance (CBM)** - Maintenance triggered by continuously monitoring component health characteristics and metrics. This allows timely intervention when needed but hinders advance planning.
4. **Predictive maintenance (PdM)** - Uses prediction tools (often statistical or machine learning models) to forecast failures and schedule maintenance proactively. PdM aims to optimise resource use, but its effectiveness depends critically on prediction accuracy; poor predictions lead to unnecessary costs or unexpected downtime.

It should be noted that several sources conflate CBM and PdM [57]. As in (Susto et al., 2012), where they are given as separate categories, we follow suit.

The PdM strategy stands out, as accurate predictive models can optimise the trade-off between improving equipment condition, reducing failure rates, and minimising maintenance costs [18]. It enables foresight for pre-emptive scheduling of large-scale maintenance. As pointed out in Section 1.2, the ISIS facility aims to strike a balance between PvM, CBM, and PdM through periods of large-scale scheduled maintenance and extensive data collection. This balance is achieved through the careful coordination between cycle scheduling, day-to-day crew-based monitoring, and the FLD [88].

In industry, many maintenance strategies prefer using PdM whilst experimenting with a variety of statistical inference and artificial intelligence modelling approaches [57, 36]. Some examples from [18] are listed in Table 1.1 which highlights the trend in the industry towards more accurate, ML-based approaches.

1.4 Data

Data relevant to the operational state and maintenance of the ISIS facility originates several sources. The ISIS team maintains proprietary internal datasets detailing specific component performance, which are available upon request. For this research, access was granted to historical logs for the ion source component, with entries dating back to 27 March 2003 and

Type	Description	Reference
Statistical	Application of SAFE to deal with PdM problems characterised by time-series data. The approach is tested on a real-life dataset of the semiconductor ion implantation process.	(Susto and Beghi, 2016)
ML	Application of SVM classification for fault prediction of rail networks, with discussion on using the model in optimising trade-offs related to maintenance schedule and costs.	(Li et al., 2014)
ML	Audio analysis on IoT devices, enabling acoustic event recognition for machine diagnosis. This paper describes designing an end-to-end system, utilising CNN-based classification.	(Pan et al., 2017)
ML	Utilisation of RF decision trees trained on SMART data to predict reliability of HDD in real-time.	(Su and Huang, 2018)

Table 1.1: Examples of applications of PdM for industrial maintenance strategies.

the Operalog, an operational log maintained by the operations crew, which documents facility faults and the corresponding remedial actions taken.

Given the breadth of available data and the project’s scope constraints, a decision was made to focus the analysis presented in this report on the **Operalog**. This dataset provides a rich textual record of real-world failures across the facility, offering valuable insights despite its partially unstructured nature. Data from the specialised ion source logs is the focus of the partner project.

1.4.1 The Operalog

The operational log, otherwise referred to as the ‘Operalog’, is an Excel spreadsheet with entries documenting moments of machinery failure within the ISIS facility covering the period 1996 - 2023. Table 1.2 documents the important features. Notably, the day-to-day operational crew have developed custom abbreviations, acronyms and terminology which may not immediately be clear. Additionally, different crew members have varying writing styles and levels of depth of information. Thus, this results in an unstructured text dataset which has a huge variance in quality and quantity of information. Furthermore, as noted in Table 1.2, **FaultRepair** was seemingly made redundant post-2017. Figure 1.4a highlights the distribution in the text lengths of both unstructured text fields. Furthermore, around 0.02% of **FaultDescription** fields are empty whereas around 66% of **FaultRepair** fields are empty. The generally shorter entries in **FaultRepair**, with a large majority having fewer than or equal to 5 characters, suggest lower informational content compared to **FaultDescription**. This is further illustrated in the Wordcloud illustrations [65] (Figure 1.5), where **FaultRepair** only has one extremely large word (‘reset’). As the size of the word indicates the frequency, this means **FaultRepair** has the word ‘reset’ in almost all non-empty entries. With ‘reset’ being exactly 5 characters and most non-empty entries in **FaultRepair** being at most 5 characters (of which, those that contain reset are over 70% of entries), it is easy to see that the **FaultDescription** field contains richer, more informative context.

Feature Name	Data Type	Description
FaultDate	Date-time	Date the fault occurred, with a precision up to the nearest second.
UserRun	String	Operational cycle that this fault has occurred within. Cycle information up to 2016 can be found at (ISIS Neutron and Muon Source, 2024) .
Downtime	Integer	The amount of time, in hours the downtime has occurred for.
Group	String, Fixed Category	The group that the faulty equipment is part of. There are 13 unique equipment groups.
Equipment	String, Fixed Category	The equipment type that failed. There are around 200 unique equipment types that have been logged to fail.
FaultDescription	String, Free-form	This is a free-form, unstructured text field that allows the on-shift operational crew to note details about the problem diagnosis and remediation that has occurred. There is no constraint to the size of this text field.
FaultRepair	String, Free-form	Another free-form, unstructured text field that allows the on-shift operational crew to note only the remediation steps. The crew seems to have stopped using this field after the end of 2017, preferring to put remediation steps in FaultDescription.
ManagersComments	String, Free-form	A very rarely used free-form text field, where the manager in charge of the on-shift crew will input comments.

Table 1.2: Description of important features (columns) in the Operalog.

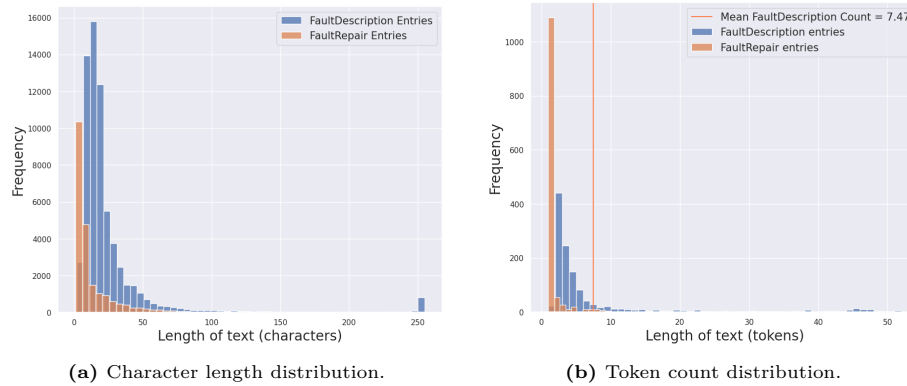


Figure 1.4: Frequency distributions comparing the non-empty `FaultDescription` and `FaultRepair` fields from the Operalog. Left shows the distribution of text lengths in characters (each bar represents a range of approximately 5 characters). Right shows the distribution of the number of tokens (each bar represents a range of approximately 2 tokens).



Figure 1.5: Comparison of word clouds for the top 300 most common words for the FaultRepair field versus the FaultDescription field. Larger words are more common and collocations are enabled so both words and phrases are shown, leading to some repeating.

Chapter 2

Natural Language Processing Background

This chapter delves into the technical background required for the methodology proposed in Chapter 3. Firstly, we discuss various measures of text similarity in Section 2.1, which motivates text embeddings and categorises similarity measures into different generations. Section 2.2 introduces the third-generation model BERT [24] and the fourth-generation universal-embedding architecture XLNet [101]. We specifically discuss the technical details of the BERT model and further iterations on it, then detail one improvement over BERT (XLNet) and outline the design principles that underpin contemporary fourth-generation models. Afterwards, we cover two methods of dimensionality reduction, PCA [67, 33] and UMAP [53], motivated by the need to visualise samples from the high-dimensional embedding spaces of the aforementioned models, in Section 2.3. Finally, we present three clustering algorithms - one partitioning-based (k-Medoids [38]) and two density-based (DBSCAN [26] and HDBSCAN [16]).

2.1 Sentence Similarity

Sentence similarity (or document similarity) is the NLP task of quantifying similarity between texts. The increasing digitisation of language motivates this task [71]. Use cases include detection of academic malpractice via plagiarism [51, 8], and text summarisation [5, 40, 37]. Similarity can be assessed lexically (based on syntactic overlap) or semantically (based on meaning) [71]. Semantic similarity, our focus, includes three main types:

- **String-based similarity** - Measures direct string similarity, accounting for character sequences and composition, using fine-grained (character-based), coarse-grained (term-based), or hybrid approaches [102].
- **Knowledge-based similarity** - Measures relatedness using semantic networks (knowledge graphs), such as the popular WordNet approach [13].

- **Corpus-based similarity** - Uses models trained on large text corpora to estimate similarity between sentence pairs. Examples include statistical models (i.e. LSA [42], SVD [81]) and early word embedding models (i.e. Word2Vec [11], GloVe [69], fastText [55]).

Corpus-based methods typically require converting text into numerical vector representations. Computing this representation involves converting unstructured text into vectors. This usually includes applying an embedding model (word or sentence level), often preceded by pre-processing steps like stop-word removal, case normalisation, or tokenisation to standardise the input [86].

This vector representation is known as an embedding, residing in an embedding space. The dimensionality of this space is the embedding dimension. This concept is important because embedding space characteristics influence the model’s ability to capture syntactic and semantic meaning, as the space encodes this information. (Bojanowski et al., 2017) shows, using Word2Vec, that different embedding dimensions produce different results. Furthermore, poorly constructed embedding spaces degrade prediction accuracy.

Therefore, semantic sentence similarity often relies directly on sentence embedding (or text embedding): the NLP task of learning effective high-dimensional vector representations. Text embedding is covered further in Section 2.2. The advent of the Transformer architecture [91] and large language models led to dominant DL approaches with high parameter counts [17] handling extremely large token sequences. While older word embedding models are often termed second-generation [17], newer generations include:

- **Third-generation** - Contextualised embeddings. These models (i.e. ELMo [75], GPT [70], BERT [24]) dynamically encode context and are often referred to as language models.
- **Fourth-generation** - Universal text embeddings. The current state-of-the-art, aiming for unified models applicable to multiple downstream tasks. Examples include Gecko [43], Multilingual E5 [96], Nomic [62].

Modern (2nd-4th gen) text embedding models are frequently used in PdM for applications such as insight extraction and clustering intents from unstructured text data [3, 90]. Such datasets in industrial applications often come from operational or managerial logs documenting failures, resolutions, and comments. Advanced models enable semi- or fully automatic insight retrieval and auto-categorisation, facilitating intuitive understanding and potentially highlighting patterns in failure [61].

2.2 Sentence Embedding

Briefly touched on in Section 2.1, sentence embedding (otherwise known as text embedding) is the NLP task of computing some high-dimensional embedding vector-space representation for unstructured text data. This vector-based representation should encode the semantic and syntactic meaning of the text and establish meaningful relationships. For example, the sentence ‘I like dogs’ should have the opposite representation to ‘I hate dogs’. However these sentences

should be more related than to the sentence ‘My house was destroyed in an earthquake’. For a naive illustration of this, see Figure 2.1, which shows how similar sentences should be grouped together. As Nomic requires the task explicitly in input text (more on this later), ‘clustering:’ is appended to all sentences. Deep learning sentence embedding models are now seen to be state-of-the-art as they are able to extract features automatically and more effectively than manual efforts, when supported with large quantities of data [49].

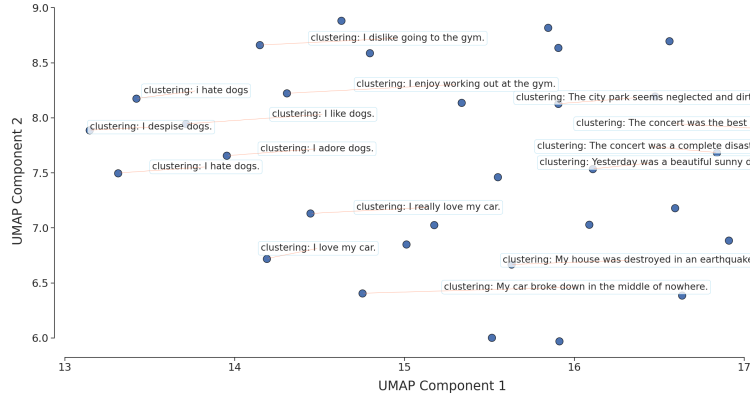


Figure 2.1: Example text embedding with Nomic Text Embedding v1.5 [62] of random sentences generated using OpenAI’s ChatGPT o3-mini-high and projected onto 2 dimensions using UMAP [53] with a random seed of 42 and minimum neighbours of 15. Only some of the labels are highlighted for visual clarity.

2.2.1 Transformers and Foundational Pre-trained Language Models

An important advancement in modern NLP is the use of Pre-trained Language Models (PLMs). Models like BERT [24] demonstrate success and improvements in performance across various NLP tasks when adapted through a process called fine-tuning [25, 56]. Fine-tuning involves taking a general-purpose (termed ‘base’) PLM, already trained on vast amounts of text data, and further training it on a smaller, task-specific dataset. This leverages the model’s pre-existing knowledge of language structure and semantics, allowing for efficient adaptation to specialised tasks like the Operalog analysis in this report. These PLMs effectively learn an embedding space for language and can often generate text, providing a powerful foundation [24]. The accessibility of these models has been greatly enhanced by platforms like HuggingFace, which host pre-trained model parameters (or ‘weights’), enabling researchers to download and utilise or fine-tune them without undertaking the computationally prohibitive initial pre-training [99]. This project leverages such pre-trained models, specifically focusing on the Transformer family.

The Transformer architecture, introduced by (Vaswani et al., 2017), employ attention mechanisms, to compute pairwise token significance. That is, conceptually, attention allows the model - when processing a word in a sequence - to weigh the importance of other words in the sequence regardless of their distance to each other. This enables the model to cap-

ture long-range dependencies and understand context more effectively than earlier sequential models. While the detailed architecture involves components like multi-head attention and layer normalisation, the core idea is this context-aware weighting mechanism [91]. Given its foundational nature and detailed coverage in the literature, we refer the reader to the original paper for architectural specifics.

Building upon the Transformer, several influential PLMs were developed. We briefly discuss two foundational models, BERT and XLNet, as their concepts and limitations motivate the subsequent choice of models like MPNet used later in this study.

BERT

The Bidirectional Encoder Representations from Transformers (BERT) model was a landmark development by Google AI [24]. Its key innovation was learning deep *bidirectional* representations by considering both the left and right context simultaneously in all layers. This contrasts with earlier models that typically processed text in only one direction. BERT achieves this primarily through its Masked Language Modelling (MLM) pre-training objective. During MLM, a percentage of input tokens (15%) are randomly masked (replaced with a special [MASK] token), and the model learns to predict these original tokens based on the surrounding unmasked context [24]. This forces the model to develop a deep understanding of word relationships and context from both directions. While BERT also used a Next Sentence Prediction task, MLM is its core contextual learning mechanism [79]. BERT demonstrated state-of-the-art performance on numerous NLP tasks after fine-tuning [24].

However, BERT has limitations. A notable issue is the pre-train-to-fine-tune discrepancy: the artificial [MASK] token used during pre-training is absent in downstream tasks (i.e. during fine-tuning), potentially hindering performance. Furthermore, by predicting masked tokens independently, BERT doesn't explicitly model the dependencies between the masked tokens themselves, which can be an oversimplification for complex language understanding, as pointed out by (Schank, 1972) [101].

XLNet

To provide context for the MPNet model used later in this project, we briefly discuss XLNet [101], a model proposed to address certain limitations of BERT while retaining the benefits of bidirectional context. XLNet employs an autoregressive (AR) approach using Permutation Language Modelling. Conceptually, instead of processing sentences in their original linear order, XLNet maximises the likelihood over all possible permutations (factorisation orders) of the input sequence. For any given permutation, it predicts a token based only on the tokens preceding it in that specific order, thereby implicitly learning context from both original directions (left and right) for every token [101]. This method avoids BERT's artificial [MASK] token and explicitly models dependencies between predicted tokens within a given permutation. It also leverages the Transformer-XL architecture for improved handling of longer sequences [21].

While XLNet offered improvements over BERT, the development of PLMs continued, leading to models like MPNet [79]. MPNet attempts to unify the strengths of both masked language modelling (like BERT) and permutation-based autoregressive modelling (like XLNet), which motivates its consideration in this project (see Chapter 3). Further technical

details regarding XLNet’s mechanisms, including the two-stream self-attention, can be found in Appendix B.1.

Considerations for Using Pre-trained Models

Finally, while leveraging PLMs significantly accelerates research, it introduces reliance on third-party models. As noted previously, users must trust that the pre-trained parameters are sound and haven’t been subject to issues like data contamination or malicious alterations. Therefore, favouring models with transparently documented data sources and training procedures is generally preferred when possible.

2.3 Dimensionality Reduction

The sentence embedding techniques discussed previously often produce high-dimensional vector representations, frequently on the scale of hundreds or even thousands of dimensions (i.e. 768 dimensions for BERT base [24]). Directly analysing or visualising data in such high-dimensional spaces is challenging. Furthermore, many data analysis methods, including the clustering algorithms discussed later (Section 2.4), can suffer from performance degradation in high dimensions due to phenomena collectively known as the ‘curse of dimensionality’ [92, 31]. This ‘curse’ refers to counter-intuitive geometric properties of high-dimensional spaces that can make concepts like distance and density less meaningful [92].

Therefore, dimensionality reduction techniques are often employed. These methods aim to find a lower-dimensional representation (typically 2-dimensional or 3-dimensional for visualisation) that captures the most important structure or variance present in the original high-dimensional data [80]. By exploiting redundancy in the data, they project points into a smaller feature space suitable for visualisation or input to subsequent algorithms like clustering [80, 31]. While numerous techniques exist, as shown by (Sorzano et al., 2014), this section focuses on two widely used methods employed in this project: Principal Component Analysis (PCA) [67, 33] and Uniform Manifold Approximation and Projection (UMAP) [53].

2.3.1 PCA

Principal Component Analysis (PCA) is a linear method for dimensionality reduction, originally developed by (Pearson, 1901) and (Hotelling, 1933). It operates by finding orthogonal axes, known as principal components, that capture the directions of maximum variance within the dataset [2]. The first principal component captures the largest amount of variance, the second captures the largest remaining variance while being orthogonal (that is, statistically uncorrelated) to the first, and so on. Projecting the original data onto the first few principal components creates a lower-dimensional representation that retains most of the original data’s variance [2].

Conceptually, PCA seeks a linear projection that either maximizes the variance of the projected data or, equivalently, minimises the ‘reconstruction error’ which is the squared distance between original points and their projections [89]. While the formal solution involves finding eigenvectors of the data’s covariance matrix, a standard and efficient method to compute

the principal components and their associated variance relies on the Singular Value Decomposition (SVD) of the (mean centred) data matrix X [81, 89]. Here, mean centred refers to $X - \frac{1}{nm} \sum_i^n \sum_j^m X_{ij}$ for $X \in \mathbb{R}^{n \times m}$. The SVD factorizes X as $X = USV^T$, where the columns of V represent the principal component directions, and the squared diagonal elements of S represent the variance captured by each component [89]. By keeping only the components associated with the largest singular values (a truncated SVD), PCA achieves dimensionality reduction while preserving maximal variance in a linear sense. A formal treatment of this as an optimisation problem can be found at Appendix B.2.3.

2.3.2 UMAP

Uniform Manifold Approximation and Projection (UMAP) is a more recent, non-linear dimensionality reduction technique based on manifold learning principles [53]. Unlike PCA's linear projections, UMAP aims to capture potentially complex, non-linear structures in the data. It operates under the assumption that the high-dimensional data lies on or near a lower-dimensional manifold embedded within the higher-dimensional space [53]. The goal is to find a low-dimensional embedding that faithfully represents the topological structure of this presumed manifold. While grounded in advanced mathematical concepts [44, 45, 98], the intuition behind the algorithm can be understood through its two main conceptual stages [53]:

1. **High-dimensional Graph Construction** - First, UMAP builds a weighted graph representation of the data in the original high-dimensional space. It identifies nearest neighbours for each data point and assigns edge weights based on the likelihood that points are closely connected on the underlying manifold. This step primarily focuses on capturing the local connectivity and structure of the data.
2. **Low-dimensional Layout Optimisation** - Then, UMAP seeks a corresponding low-dimensional embedding where the graph structure is as similar in structure as possible to the high-dimensional graph. It iteratively adjusts the positions of points in the low-dimensional space to minimise the difference (often measured using cross-entropy, a measure of divergence between probability distributions [104]) between the connectivity probabilities in the high- and low-dimensional representations. This optimisation aims to keep points connected in the low-dimensional space if they were connected in the high-dimensional space, and separated if they were not.

The resulting low-dimensional embedding aims to preserve both local neighbourhood structure and, importantly, aspects of the data's broader global structure [53]. UMAP's output is influenced by several key hyperparameters:

- **n_neighbors** - Controls the size of the local neighbourhood considered. Lower values emphasize local structure, potentially revealing fine clusters, while higher values capture more global structure, potentially merging related groups [53].
- **min_dist** - Affects the minimum separation between points in the low-dimensional embedding. Lower values create tighter, denser clusters, while higher values produce more spread out embeddings where intra-cluster structure might be more visible [53].

- **n_components** - Defines the dimensionality of the output embedding (typically 2- or 3-dimensions for visualisation).
- **metric** - Specifies the distance measure used in the high-dimensional space (i.e. 'euclidean', 'cosine'). The choice is crucial, 'cosine' distance is often preferred for high-dimensional text embeddings as it measures orientation rather than magnitude [53].

Selecting appropriate hyperparameters often requires experimentation (as explored in Section 3.6). UMAP implementations are generally computationally efficient compared to other non-linear methods like t-SNE [14, 53], making it practical for many applications. Figure 2.2 provides a visual intuition of UMAP applied to structured data (sampled points from the skeletal structure of an elephant). Notably, structure preservation can be visually observed, as points from a tusk represent a shape which mirrors a tusk shadow in the 2-dimensional down-projection. However, while powerful for revealing non-linear structures, the axes of a UMAP projection do not have the direct variance interpretation of PCA's principal components.

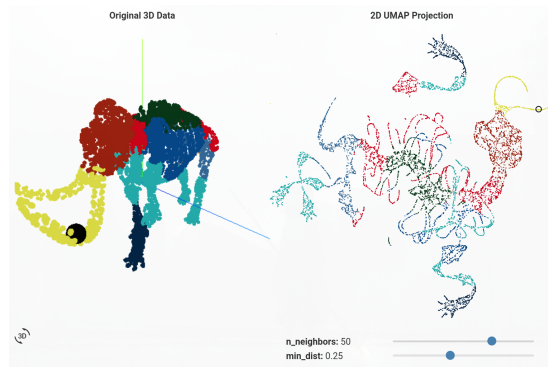


Figure 2.2: An illustration of UMAP applied (right) on samples taken from the skeletal structure of an elephant in 3-dimensions (left) with the UMAP hyperparameter values displayed. Highlighted, a black sphere (left) and circle (right), shows a chosen point mapped from 3-dimensions to 2. Source: (Coenen, Andy and Pearce, Adam, 2020).

2.4 Clustering

Clustering is an unsupervised machine learning task that aims to group data points such that items within a group (a cluster) are more similar to each other than to those in other clusters. Common approaches include partitioning methods, which divide the data into a predefined number of clusters, density-based methods, which identify clusters based on regions of high data point density, and hierarchical methods, which build a tree-like structure of nested clusters [39].

This section provides the necessary background for the clustering techniques employed later in this project. We will cover k-Medoids [38], a partitioning algorithm; Density-Based

Spatial Clustering of Applications with Noise (DBSCAN) [26], a density-based algorithm; and Hierarchical DBSCAN (HDBSCAN) [16], which builds upon density and hierarchical concepts.

2.4.1 k-Medoids

The k-Medoids algorithm is a partitioning-based clustering method that aims to divide a dataset into k distinct clusters. The k-Medoids algorithm uses actual data points, known as medoids, as the centres of its clusters. It aims to minimise a cost function based on the sum of pairwise dissimilarities (that is, distances) between each point and the medoid of the cluster it is labelled under [38].

A classical algorithm for implementing k-Medoids is Partitioning Around Medoids (PAM), introduced by (Kaufman and Rousseeuw, 1990). PAM operates in two main phases:

1. **The build phase** - An initial set of k data points are selected as the starting medoids. This selection aims to establish a reasonable initial clustering by choosing points that are relatively central within potential groups that exists, minimising the initial total dissimilarity.
2. **The swap phase** - This phase iteratively refines the set of medoids. For each currently selected medoid m and each non-medoid point data point x , the algorithm computes the change in total dissimilarity that would result from swapping m and x . That is, setting x as that centroid of m 's cluster. The swap that yields the greatest reduction across all possible pairs is finally performed. This process repeats until no swap can further reduce the total dissimilarity which indicates a local optimum has been reached.

The PAM algorithm employs a greedy search strategy during the swap phase, which means it may converge to a local rather than a global optimum [38]. The computational complexity of the swap phase is often cited as $O(k(n - k)^2)$ per iteration, where n is the number of data points and k is the number of clusters [38].

Appendix B.3 provides a high-level pseudocode description of the relatively straightforward PAM process. It takes the desired number of clusters k and the dataset X as input, and outputs the final set of medoids M and set of clusters X' . The fact that k is a parameter to the algorithm means that either the number of clusters must be known beforehand or must be found through hyperparameter optimisation.

2.4.2 DBSCAN

Density-Based Spatial Clustering of Applications with Noise (DBSCAN) is density-based clustering algorithm, unlike partitioning methods like k-Medoids [26]. Instead of partitioning data around centres, DBSCAN groups points in high-density regions, separated by sparser areas. Key characteristics are its ability to find arbitrarily shaped clusters and potentially identify noise, without requiring a predefined number of clusters [26]. The core intuition is that points belonging to the same conceptual group tend to lie close to each other, forming denser regions than noise or outliers.

The algorithm's behaviour and the concept of density is controlled by two hyperparameters [26]:

- **Eps**: Defines the maximum distance (neighbourhood radius) between two points to be considered neighbours. It essentially sets the radius for identifying neighbours; lower values require points to form dense regions.
- **MinPts**: Specifies the minimum number of points required within a point’s Eps-neighbourhood for it to be considered a **core point** (an interior point of a dense region). This defines the threshold for density; higher values may result in lower clusters with more points classified as noise.

Determining appropriate values for **Eps** and **MinPts** often requires a heuristic (such as the one described by (Ester et al., 1996)) or domain knowledge, as DBSCAN is an unsupervised machine learning algorithm [26].

Key concepts include the **Eps-neighbourhood** (points within radius **Eps**), **core points** (points with at least **MinPts** neighbours within **Eps**), **border points** (non-core points reachable from a core point), and **noise points** (neither core nor border). A point q is **density-reachable** from a core point p if they are connected by a chain of core points within their respective Eps-neighbourhoods [26].

To understand the algorithm’s operation, several core concepts are essential [26]

- **Eps-neighbourhood** - Points within radius of **Eps**.
- **Density-reachable** - Point with a chain of core points, with the greatest distance between any pairs of points being less than or equal to **Eps**.

Additionally, based on the hyperparameters, DBSCAN categorises each point in the dataset into one of the three types [26].

- **Core point** - A point whose Eps-neighbourhood contains at least **MinPts** points, including itself. Core points are considered to be in the interior of a dense cluster.
- **Border point** - A point that is not a core point itself, but falls within the Eps-neighbourhood of at least one core point. Border points lie on the border of a cluster.
- **Noise point** - A point that is neither a core point nor a border point. These points typically reside in low-density regions and are not assigned to any cluster.

Conceptually, the algorithm operates by selecting an arbitrary unvisited point. If it is identified as a core point, a new cluster is initiated. This cluster is then expanded to include all points density-reachable from it (both core and border points), effectively following chains of neighbouring core points. Points not reachable from any core point are designated as noise [26]. Note that the assignment of border points reachable by multiple clusters can be implementation-dependent but typically is assigned by the first cluster to reach it [26].

In general, DBSCAN offers significant advantages: it can identify clusters of arbitrary shape, is inherently robust to noise, and does not require the number of clusters to be specified beforehand. However, it also has limitations: its performance is sensitive to the choice of **Eps** and **MinPts**, and it can struggle with clusters of significantly varying densities [26, 16]. With appropriate index structures, its average runtime complexity is typically $O(n \log n)$ for n data points [26]. Like many distance-based algorithms, its effectiveness can degrade in very high-dimensional spaces due to the ‘curse of dimensionality’ (Section 2.3) [31, 92].

2.4.3 HDBSCAN

Hierarchical Density-Based Spatial Clustering of Applications with Noise (HDBSCAN) [16] advances upon DBSCAN by generating a hierarchical clustering structure. It addresses DBSCAN’s reliance on a single density threshold (ϵ), improving identification of varying density clusters by effectively exploring all density levels ($\lambda = \frac{1}{\epsilon}$, for all $\epsilon \in [0, \infty)$) to build its hierarchy [16].

Key contributions are the hierarchy construction algorithm (producing a dendrogram) and a novel ‘cluster stability’ measure used to extract an optimal flat partition containing the most persistent clusters [16]. Extraction optimises total stability. Advantageously, HDBSCAN mainly requires only the m_{pts} parameter (equivalent to DBSCAN’s `MinPts`), whose effect as a density smoothing factor is relatively well-understood [16].

Importantly, the following discussion assumes basic familiarity with graph theory; (Harris, 2008) provides a primer.

Theoretically, HDBSCAN relates to DBSCAN by viewing clusters as connected components of core points within a graph, where edges connect mutually reachable core points (based on ϵ, m_{pts}), above varying density levels [16]. It utilises concepts derived relative to m_{pts} [16]:

- **Core distance** ($d_{\text{core}}(x)$): Distance to the m_{pts} -th nearest neighbour, measuring local density (lower is denser).
- **Mutual reachability distance** ($d_{\text{mreach}}(x_p, x_q)$): Defined as

$$\max\{d_{\text{core}}(x_p), d_{\text{core}}(x_q), d(x_p, x_q)\}$$

This metric effectively increases distance between points in sparser regions.

The algorithm constructs the hierarchy conceptually by first computing core distances, then building a graph weighted by mutual reachability distance, finding its Minimum Spanning Tree (MST), and deriving the hierarchy (dendrogram) from the sorted MST edge weights. This captures cluster structures across all density levels [16].

To extract a meaningful flat partition, HDBSCAN introduces ‘cluster stability’. This quantifies how persistent a cluster C_i is across the range of density levels ($\lambda = \frac{1}{\epsilon}$). Stability $S(C_i)$ is calculated for each cluster by summing, for each point x_j belonging to C_i , the range of λ values for which x_j remains part of that specific cluster [16]. More formally, using $\lambda_{\min}(C_i)$ (threshold where density point where cluster appears) and $\lambda_{\max}(x_j, C_i)$ (threshold where density point leaves cluster):

$$S(C_i) = \sum_{x_j \in C_i} (\lambda_{\max}(x_j, C_i) - \lambda_{\min}(C_i)) \quad (2.1)$$

Equation 2.1 adapts the concept of ‘excess of mass’ (i.e. the total density in a cluster after it is formed) [59] to a set of discrete data points and density levels derived from the hierarchy.

Clusters where points persist over a larger range of λ values (i.e. are more ‘stable’) will have higher stability scores.

Using this stability measure, HDBSCAN selects the optimal flat clustering by finding a set of disjoint clusters from the hierarchy that maximizes total stability. This is achieved efficiently by traversing a simplified cluster tree (pruned using a minimum cluster size, $m_{cluster_size}$, typically set to m_{pts}), effectively performing optimal ‘cuts’ at different hierarchy levels based on cluster persistence [16].

In summary, by building a full hierarchy and selecting clusters based on persistence across density levels, HDBSCAN has been shown to be more robust than DBSCAN and excels at identifying clusters of varying densities. Its primary disadvantages can be potential sensitivity to the chosen distance metric and the m_{pts} parameter [16].

Chapter 3

Automatic Categorisation and Label Generation

3.1 Overview

This project transforms unlabelled Operalog data into a categorised dataset through a five-step pipeline (Figure 3.1): (1) pre-processing and normalising the **FaultDescription** text to reduce noise; (2) generating high-dimensional semantic embeddings to capture meaning numerically; (3) projecting embeddings to a lower dimension using UMAP while preserving (topological) structure in the original high-dimensional space; (4) clustering the low-dimensional data, with configurations optimised using internal validation heuristics to group similar descriptions automatically; and (5) inducing descriptive labels for the discovered clusters using NLP techniques. The analysis focuses solely on the **FaultDescription** field, as the **FaultRepair** field generally lacks sufficient informational content, with the majority either empty or less than 6 characters long. A diagram of the pipeline is shown in Figure 3.1.

Several hyperparameters affect the performance of the label generating process. Examples include model or algorithm specific parameters (i.e. the dimensionality reduction or clustering algorithms) and clustering ‘goodness’ heuristics used. These hyperparameters are tuned using Optuna [4], an automatic optimisation framework specifically designed for machine learning applications and allows for multi-objective optimisation. This hyperparameter optimisation process determines ‘best’ induced categories, and thus labels, for each issue type.

Additionally, as this project is an early stage exploration of auto-categorisation for the ISIS Operalog, the focus is on entries of ion source **Equipment** type. This enables the project to focus on refining the approach in a scoped manner with a structured goal in mind - to progress the auto-categorisation effort on the ISIS Operalog. The subsequent sections of this chapter detail each stage of this end-to-end pipeline, developed with the goal of producing an effective auto-categorisation model for the Operalog data.

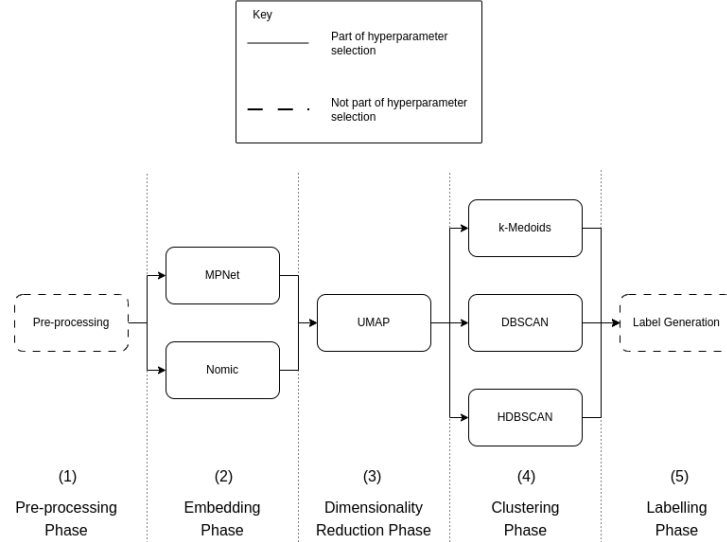


Figure 3.1: An overview of the end-to-end machine learning pipeline, outlining each key stage and highlighting which sections required hyperparameter optimisation.

3.2 Natural Language Pre-processing

The aim of pre-processing the unstructured, English natural language text data from **FaultDescription** is to normalise and prepare it for text embedding. Normalisation allows many syntactic permutations or augmentations of a text to be collapsed into one standardised form. This allows for consistent results as it ensures texts that are syntactically different but semantically similar will be mapped by embedding model to the same value. For pre-processing, and the rest of the pipeline, each log entry’s **FaultDescription** is considered one text.

Tokenisation is one of the earliest stages of natural language processing [86, 74]. To perform tokenisation, we use Tensorflow’s **UnicodeScriptTokenizer** [1]. Tensorflow is ‘an end-to-end platform for machine learning’, which provides efficient machine learning algorithm implementations. A tokeniser, such as this, splits the text into sub-units called ‘tokens’ [29]. These tokens are useful to pass to subsequent natural language processing stages which is, in our case, text embedding. **UnicodeScriptTokenizer**, a specialised version of Tensorflow’s **Tokenizer**, tokenises ‘UTF-8 strings by splitting where there is a change in Unicode script’ [1].

Once tokenised, stop-word removal is performed. Stop-words are non-informative words such as articles, prepositions and pronouns. Therefore removal of these words tends to help models have access to informative contexts and reduce noise [78]. We consider the standard English stop-words from NLTK [10] and some context specific stop-words. This includes ‘ion source’ which does not provide any informative contexts as we have already scoped our dataset to ion source **Equipment** and ‘breakdown’ which appears in just over 50% of entries.

As mentioned previously, there are a few non-trivial abbreviations used by the operational crew when writing a log entry. These abbreviations are normalised into a standard English word or phrase (see Table 3.1).

Furthermore, punctuation tokens are converted into one of two categories: end-of-sentence (EOS) tokens or regular punctuation (PUNC) tokens. The goal for this is to normalise punctuation to either signify sentence boundaries or collapse into one token. It is not trivial to see whether the **FaultDescription** text utilises punctuation to convey additional meaning. Thus, the step was made parametrisable via a flag which decides whether punctuation mapping is enabled. Similarly, text casing normalisation (converting all text to lower-case) follows the same procedure.

Just in terms of ion source **Equipment**, the pre-processing is applied across 1251 log entries, ranging from 2009 - 2023. Tokenisation and operating on tokens is computationally intensive, thus the solution operates on tensors, leveraging Tensorflow capabilities to perform GPU-based compute, which speeds up machine learning based applications [1, 9].

To improve computational efficiency, Boolean flags controlling text casing and punctuation removal were excluded from the hyperparameter optimisation phase (Section 3.6). Incorporating them would require the computationally expensive pre-processing step to be repeated for each Optuna [4] trial, a hyperparameter framework used in Section 3.6, even for minor parameter adjustments unrelated to these flags. Given that there are only a few combinations for these Boolean settings, they are more efficiently managed as user-selectable options via the CLI application (Section 3.8), ensuring pre-processing is performed only once per selected configuration.

Abbreviation Regular Expression	Mapped word or phrase
o/p	output
i/s	ion source
(b/down break-down b\down b/d)	breakdown

Table 3.1: Operational crew abbreviation mapping.

3.3 Text Embedding Model Selection and Application

After pre-processing, the cleaned and normalised **FaultDescription** text requires transformation into a numerical format suitable for analysis. Sentence embedding (introduced in Section 2.2) provides this by mapping text into a high-dimensional vector space, where semantic relationships are preserved. This is crucial for the project’s goal: automatically categorising log entries by grouping semantically similar entries, which corresponds to finding similar fault types.

3.3.1 Dataset and Hardware Driven Requirements

Effective categorisation relies on an embedding model that can accurately capture the syntactic and semantic nuances of these technical descriptions. Figure 1.4b highlights that, although

the average **FaultDescription** entry has roughly 7 tokens, in the dataset, over 60% of entries have less than 5 tokens and around 10% of the 1251 entries have over 20 tokens. This includes entries which stretch to multiple sentences. Additionally, the vocabulary is heavily domain specific (i.e. ‘ion source’) and a single out-of-place adjective can reverse the meaning of a fault report (i.e. ‘beam loss’ versus ‘no beam loss’).

Furthermore, the hardware available for research is detailed in Table 3.2. Some models, such as large language models with a parameter counts in hundreds of millions or billions may exceed the 6GB GPU VRAM limit during inference. Therefore, we restrict consideration to models whose inference memory footprint and compute latency remain feasible on this hardware.

As a result, we require a model that (1) handles sequences that contain both long- and short-term dependencies but still establish semantic relationships effectively; (2) is able to capture the semantic contextual meaning from the entire sequence - that is, bidirectionally; and (3) is able to be feasibly loaded and run on the research hardware specified in Table 3.2.

Hardware	Description
CPU	11th Gen Intel i7-11800H (16 core) @ 4.600GHz
RAM	64GB
GPU	NVIDIA GeForce RTX 3060 Mobile / Max-Q
GPU VRAM	6GB

Table 3.2: Research hardware description.

3.3.2 Candidate Models

With these constraints in mind, we look at the recent state-of-the-art embedding models and highlight models that were not in consideration. Candidate models were compared on the constraints above and selected based on pre-fine-tuned models published on HuggingFace (so we can use them directly, without re-training) [99]. HuggingFace is an open-source platform for machine learning related development, where researchers can upload pre-trained and fine-tuned model parameters. The models that ranked high on clustering tasks on the Massive Text Embedding Benchmark (MTEB) leaderboard, which compares more than 100 text and image embedding models across 132 tasks of 9 categories - 17 of which are clustering tasks [58].

Two BERT-family sentence-embedding models satisfy the constraints above, MPNet and Nomic. Both models were used with their out-of-the-box HuggingFace settings (i.e. maximum token lengths of 384 and 8192, respectively and embedding dimension of 768) [79, 62, 63].

MPNet - an overview 512-token context window with a 768 embedding dimension. Unifies BERT’s masked and XLNet’s permutation language modelling pre-training objectives. This gives better positional awareness for tokens that are further away from each other [79]. MPNet has around 109 million parameters, meaning it is able to load onto the research hardware. The HuggingFace model path is: `sentence-transformers/all-mpnet-base-v2` and achieves a clustering score of 40.77, ranking at 98 on the MTEB leaderboard, at time of writing.

Nomic - an overview Current state-of-the-art 8192-token context window with up to a 768 embedding dimension. Utilises rotary positional embeddings and flash attention that provide long-context support while remaining relatively small, with around 137 million parameters. Task prefixes (i.e. `classification:`, `clustering:`, etc.) encourage distinct semantic sub-spaces, thus we prepend all inputs passed to this model with ‘clustering:’. Specifically, we use version 1.5 which takes advantage of Matryoshka Representation Learning (MRL) and is able to encode coarse versus fine-grained semantic information in embedding vector sub-spaces [62, 63]. The HuggingFace model path is: `nomic-ai/nomic-embed-text-v1.5` and achieves a clustering score of 41.55, ranking at 54 on the MTEB leaderboard, at time of writing.

Excluded Models The following embedding models were evaluated but ultimately excluded, either because they exceeded our 6GB VRAM budget or failed to meet clustering performance thresholds in pilot runs (see Table 3.3). Excluding these ensures all retained models both fit our hardware and deliver the minimum clustering quality we require.

Model	Parameters	Exclusion Reason	Reference
Qwen2-7B	7 billion	Requires more than 6 GB VRAM at inference.	(Yang et al., 2024)
Mistral-7B	7 billion	Similar VRAM overrun on consumer GPUs.	(Wang et al., 2023)
e5-large	440 million	Scored only 32 points on MTEB’s clustering benchmark.	(Wang et al., 2024)

Table 3.3: Models considered but excluded from the embedding pipeline.

Detailed Rationale

In this section, we elaborate on the rationale for selecting MPNet and Nomic as the primary embedding models for evaluation, connecting their specific properties to the requirements identified in Section 3.3.1 and the conceptual background from Chapter 2.

MPNet Justification MPNet [79] was for its hybrid pre-training approach, combining masked and permutation language modelling benefits to potentially mitigate drawbacks of earlier methods like BERT [24] and XLNet [101]. This hybrid nature seemed particularly well-suited to the Operalog dataset for several reasons.

1. MPNet’s unified objective learns bidirectional context while modelling token dependencies [79], which appeared suitable for the Operalog’s mix of short and long entries (Section 3.3.1). This capability is potentially advantageous for capturing relationships between technical terms that might be separated within longer **FaultDescriptions**. The formal definition of MPNet’s objective function and an example of its input structuring are provided in Appendix B.2 for completeness (see Equations B.6 and B.7).
2. MPNet’s modified attention mechanism aims to reduce the pre-train and fine-tune discrepancy, compared to earlier models [79]. Allowing the model component responsible for prediction, query stream (see Appendix B.1.2), access to the full sequence during pre-training potentially aligns better with its use for direct embedding generation as an off-the-shelf model in this project. This will aid long-range dependency capture.

3. MPNet demonstrated strong performance on various NLP benchmarks at the time of its release, including the GLUE benchmark [94, 79], suggesting a robust general language understanding capability applicable to the technical language in the Operalog. Its parameter count (approximately 109 million) also made it feasible for the available hardware (Table 3.2).

Nomic Justification Nomic `nomic-embed-text-v1.5` [62, 63], was selected as a recent state-of-the-art open-source embedding models, offering several potentially beneficial features:

1. Its design for long contexts (up to 8192 tokens), using Rotary Positional Embeddings (RoPE) [83] is advantageous given the significantly variable lengths of Operalog entries (Figure 1.4), potentially offering better representation for detailed logs compared to MPNet’s 512-token window [79].
2. Despite its long context, Nomic’s computational efficiency, achieved through a moderate parameter count (roughly 137 million) and optimisations like FlashAttention [22] (reducing memory usage and latency), ensured feasibility within the project’s 6GB VRAM constraint (Table 3.2), unlike large models (Table 3.3).
3. Nomic also offers task-specific optimisation via input prefixes [62]. By prepending ‘`clustering:`’ to each `FaultDescription` entry, we aimed to leverage specific semantic subspaces within the model potentially optimised during its pre-training for clustering tasks. This was hypothesised to generate embeddings more suitable with the primary goal of grouping similar fault types compared to using a generic embedding.
4. Nomic v1.5 incorporates Matryoshka Representation Learning (MRL) [41, 63]. While this project utilised the full 768-dimension embeddings, MRL imbues the embedding space with a nested structure where shorter prefixes remain meaningful. This structured property might implicitly aid downstream tasks like clustering by organizing semantic information hierarchically, although exploring different MRL embedding lengths was outside the scope of this project.
5. Nomic’s demonstrated strong performance for its size on benchmarks like MTEB, including clustering tasks, provided empirical validation for its consideration [58, 64].

3.3.3 Section Summary

By evaluating both MPNet (representing a mature, unified BERT/XLNet approach) and Nomic (representing recent advancements in long-context efficiency and specialised embeddings), which meet the requirements criteria set in Section 3.3.1, this project aimed to explore different effective strategies for representing the challenging Operalog text data.

3.4 Embedding Dimensionality Reduction

The high dimensionality (768 dimensions) of the text embeddings necessitates dimensionality reduction, initially to enable visualisation and subsequently to aid clustering (referring to the

‘curse of dimensionality’ [92], see Section 2.3 for more details). This section explores and justifies the applicability of two techniques for this purpose: Principal Component Analysis (PCA) [67, 33], a linear method, and Uniform Manifold Approximation and Projection (UMAP) [53], a non-linear manifold learning approach. While another non-linear technique, t-distributed Stochastic Neighbour Embedding (t-SNE), was initially considered, we focused on the evaluation of PCA and UMAP. This decision was based on UMAP’s reported advantages in preserving global data structure more effectively than t-SNE, alongside its generally greater computational efficiency and run-to-run consistency [53]. Given the project’s scope, these practical benefits drew us away from t-SNE. The following subsections present a comparison between PCA and UMAP based on visualisations, justify the subsequent selection of UMAP for our pipeline; and detail its hyperparameter application to the Operalog dataset.

3.4.1 Visualisation

An example visualisation comparing the 3-dimensional projection, obtained using both PCA and UMAP in both MPNet and Nomic embedding spaces, is shown in Figure 3.3. Since the original data resides in a 768-dimension embedding space, direct visualisation is impossible. Projecting this data down to the three dimensions enables visualisation but potentially loses information, making purely visual comparisons between the method outputs subjective.

To supplement the visual assessment with a quantitative measure, we evaluate the spread within these 3-dimensional projections using normalised variance. Normalised variance, defined for a data matrix X in Equation 3.1, serves as a metric to quantify the overall dispersion of the points in the low-dimensional space. Observing Figure 3.3, the UMAP projections visually appear to exhibit more defined structures, such as clearer separations and groupings of points. Comparatively, the corresponding PCA projections look more dispersed and do not show any ‘clusters’ or groupings of points. This visual impression is supported quantitatively by the normalised variance metric, calculated for these specific runs. The values for the UMAP projections (0.90 for MPNet, 2.27 for Nomic) are substantially lower and closer to 1 than those for PCA (approximately 1.0×10^7 for MPNet, -2.4×10^9 for Nomic). In this context, the lower normalised variance for UMAP suggests a projection that arranges the data more compactly while potentially preserving meaningful neighbourhood relationships, consistent with UMAP’s goal of modelling the data manifold.

$$\text{Normalised Variance}(X) = \frac{\text{Var}(X)}{\text{Mean}(X)} \quad (3.1)$$

However, as discussed in Section 2.3.2, UMAP is a stochastic algorithm. That is, due to an element of algorithmic randomness, its output naturally varies between runs [53]. To account for this variability, Figure 3.2 visualises the distribution of normalised variance results obtained over 100 independent UMAP runs for both embedding spaces. While variation exists, these distributions reaffirm the concept that UMAP consistently produces projections with significantly lower normalised variance than those produced by PCA. The distributions exhibit some right-skewness, particularly for the MPNet embeddings but remain concentrated in a range indicative of more structured projections than those from PCA.

Based on the combination of visually apparent characteristics in the example projections Figure 3.3 and the consistently lower (although variable) normalised variance scores, indicative of the better preservation of manifold structure, in Figure 3.2, UMAP is selected as the dimensionality reduction technique for the remainder of this pipeline. It appears more likely, than PCA, to produce a low-dimensional representation that retains structural information beneficial for the next clustering stage.

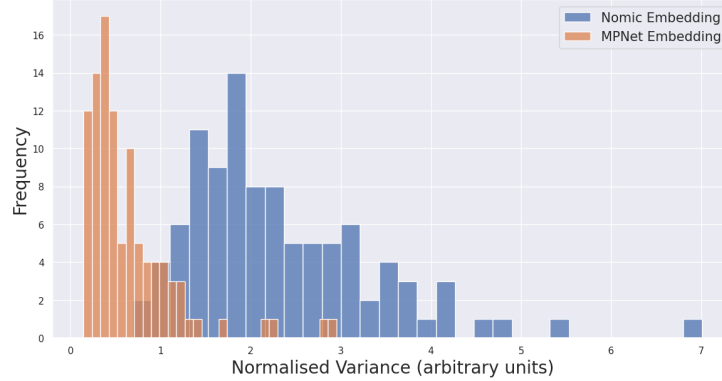


Figure 3.2: Histogram plot of 100 UMAP runs’ normalised variance (Equation 3.1) is shown for both MPNet and Nomic embedding spaces of the Operalog `FaultDescription` entries. UMAP parameters: `min_dist` = 0.1, `n_neighbors` = 15, `metric` = cosine.

3.4.2 UMAP Hyperparameters

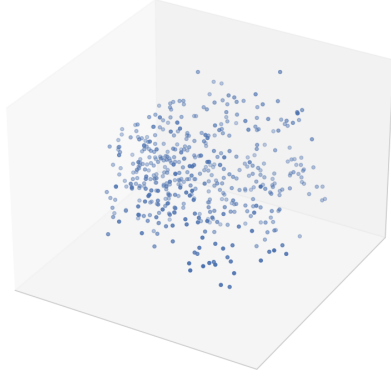
As discussed in Section 2.3.2, tuning UMAP’s hyperparameters is crucial for obtaining a meaningful low-dimensional representation of the high-dimensional Operalog text embeddings. The choice of parameters like `n_neighbors` and `min_dist`, along with the distance `metric`, significantly influences the resulting structure.

Based on the nature of the Operalog data, adjusting `n_neighbors` involves a trade-off. Lower values might isolate highly specific or rare fault types but could cause fragmentation of broader categories. Higher values might group related issues more globally but risk merging unique sub-types. Given the mix of common and rare faults, exploring a range of neighbour values is necessary.

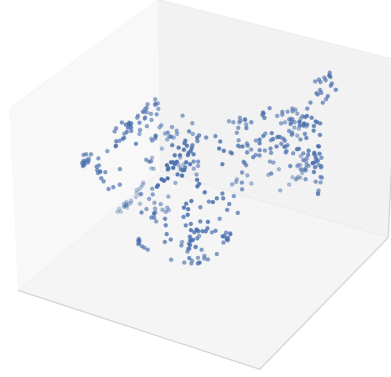
Similarly, `min_dist` affects the visual density. Here, lower values may create tightly packed, visually distinct clusters suitable for identifying major categories. However, higher values might reveal finer intra-cluster variations relevant for understanding subtypes, but could make overall separation less clear. This parameter will affect clustering algorithms, such as DBSCAN as it essentially determines potential cluster density [26]. The choice of `metric` (i.e. ‘cosine’ vs. ‘euclidean’) is also critical, as ‘cosine’ is often better suited for high-dimensional text embeddings like those used here [8, 17].

To gain an initial understanding of suitable UMAP parameters for the Operalog data, a naive grid search was performed using both MPNet and Nomic embeddings across the parameter ranges displayed in Table 3.4. The resulting visualisations for each parameter combination

Method: PCA | Normalised variance: 10121970.43

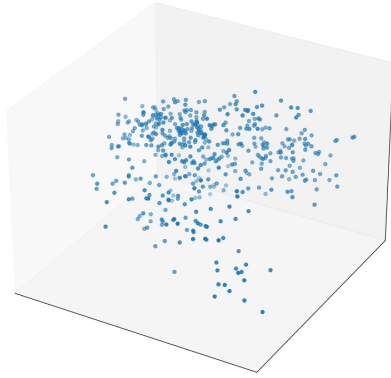


Method: UMAP | Normalised variance: 0.90



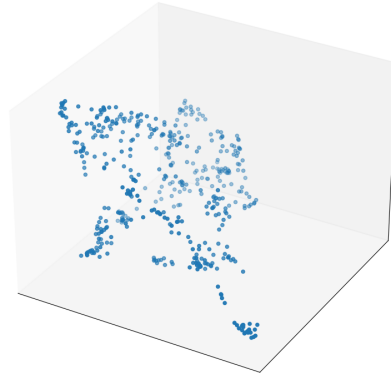
(a) MPNet — PCA

Method: PCA | Normalised variance: -2424698857.76



(b) MPNet — UMAP

Method: UMAP | Normalised variance: 2.27



(c) Nomic — PCA

(d) Nomic — UMAP

Figure 3.3: Comparison of PCA versus UMAP dimensionality reduction on the MPNet (top row) and Nomic (bottom row) embedding spaces on Operalog `FaultDescription` entries Normalised variance (Equation 3.1) is shown. UMAP parameters: `min_dist` = 0.1, `n_neighbors` = 15, `metric` = cosine.

are presented in Appendix A.1. However, visual inspection of these results highlighted a surprising finding: contrary to common practice for text embeddings, cosine distance did not appear to yield projections demonstrably better at preserving structure more faithfully than Euclidean distance in this preliminary exploration. These inconclusive and unexpected outcomes underscore the need for a more systematic hyperparameter optimisation phase, which is later detailed in Section 3.6.

Hyperparameter	Values Explored
n_neighbors	{ 5, 10, 13, 15, 20, 25 }
min_dist	{ 0.0125, 0.05, 0.1, 0.15, 0.2 }
metric	{ 'euclidean', 'cosine' }

Table 3.4: UMAP hyperparameter values explored in naive grid search

3.4.3 Section Summary

Based on the comparison in the previous subsections, UMAP was selected over PCA as the preferred dimensionality reduction technique for the MPNet and Nomic embedding spaces. The preliminary visualisations also highlighted that UMAP’s output is sensitive to its hyperparameters, indicating that a systematic approach is required to find optimal settings for revealing structure within the Operalog data. Directly evaluating the ‘quality’ of different UMAP projections is challenging therefore, to assess the effectiveness of various projections (resulting from different hyperparameter settings), we adopt an indirect evaluation strategy by applying clustering algorithms to the low-dimensional UMAP outputs. The quality of the resulting clusters, measured using specific clustering evaluation metrics (introduced in the subsequent Section 3.5), will serve as a proxy for how well the UMAP projection preserved the relevant structure for categorisation. The next section introduces the clustering algorithms and metrics chosen for this purpose

3.5 Unsupervised Categorisation Through Clustering

Following dimensionality reduction via UMAP, the resulting low-dimensional embedding space is prepared for the core task of clustering. As outlined in Section 2.4, clustering is an unsupervised machine learning approach used to group similar data points. Within this project’s context, the goal is to group the embeddings of Operalog `FaultDescription` entries such that each resulting cluster corresponds to a distinct, underlying fault category. Ideally, the clustering process should automatically discover these latent categories from the data structure.

To explore different clustering paradigms, three algorithms were selected for evaluation based on their characteristics described in Section 2.4:

- **k-Medoids** [38] - A partitioning-based algorithm that groups data around actual data points (medoids).

- **DBSCAN** [26] - A density-based algorithm that identifies clusters based on regions of high point density.
- **HDBSCAN** [16] - An algorithm combining density-based principles with hierarchical clustering to identify stable clusters across varying densities.

Notably, k-Means [52], another common partitioning algorithm, was considered but excluded due to its known sensitivity to initialisation and potential instability compared to k-Medoids [103].

A fundamental challenge in unsupervised clustering is evaluating the quality of the resulting clusters without ground truth labels. To quantitatively assess and compare the different clustering outcomes, three established internal evaluation metrics (heuristics) are employed: the Silhouette coefficient [72], the Davies-Bouldin index [23], and the Calinski-Harabasz index [15].

This section details the rationale for selecting the specific clustering algorithms and introduces the evaluation heuristics used in the subsequent hyperparameter optimisation phase.

3.5.1 Clustering Candidates

The suitability of each clustering algorithm depends on the structure of the data and the nature of the desired clusters. Below, we discuss the characteristics of each chosen algorithm in the context of the Operalog embedding data.

k-Medoids As described in Section 2.4.1, the k-Medoids algorithm (implemented via the PAM algorithm in common libraries like scikit-learn [68]) requires only one primary hyperparameter: k , the desired number of clusters. While determining the optimal k for the unknown Operalog fault categories is non-trivial, having a single parameter simplifies the optimisation process. Furthermore, k has an intuitive interpretation: higher k might isolate rare but specific ion source faults, while lower k might group broader issues such as ‘ion source failure’. However, k-Medoids assumes clusters are roughly spherical and aims to minimise distances to central medoids [38]. Visual inspection of the UMAP projections (Figure 3.3 and Appendix A.1) suggests the Operalog embeddings form complex, non-spherical structures. Therefore, k-Medoids might struggle to accurately capture the boundaries of these potential fault categories.

DBSCAN This algorithm (Section 2.4.2) uses two key hyperparameters, **Eps** and **MinPts**, to define local density. Its main advantages are the ability to find arbitrarily shaped clusters and explicitly identify noise points, which could be valuable for isolating unusual or poorly described Operalog entries [26]. However, DBSCAN’s performance relies heavily on selecting appropriate **Eps** and **MinPts** values. Its use of a single, global density threshold makes it potentially sensitive to datasets containing clusters of varying densities [16]. Assessing whether the Operalog embeddings exhibit such density variations is difficult visually. The IsoScore metric [73], which measures the uniformity of space utilisation by the projected points, provides some insight. Figure 3.4 shows average IsoScores of 0.653 (MPNet) and 0.710 (Nomic) for the UMAP projections. While these values indicate reasonable space utilisation, they are not

close to 1, suggesting potential density variations that might pose a challenge for DBSCAN’s global threshold approach. We expect some very short `FaultDescription` entries which are common, leading to dense clusters and longer entries, which are more rare, leading to sparse clusters. This insight would support the concern about the varying densities.

HDBSCAN As an extension of DBSCAN (Section 2.4.3), HDBSCAN aims to overcome the sensitivity to Eps and the varying density problem. It achieves this by building a cluster hierarchy and using a stability measure to extract clusters that persist across a range of density levels [16]. It requires only one primary hyperparameter, m_{pts} (equivalent to `MinPts` in DBSCAN), making it simpler to tune than DBSCAN. Its ability to handle varying densities and its robustness (consistent results between runs) make it a strong candidate for the Operalog data, where the true number and density of fault categories are unknown. As mentioned before, since we expect a clusters of varied densities, being able to find stable clusters in this space is desirable for discovering potentially diverse fault types in the Operalog.

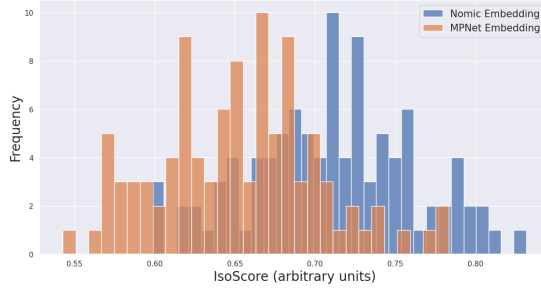


Figure 3.4: IsoScore comparison of MPNet versus Nomic embeddings over 100 runs of UMAP. UMAP parameters: `min_dist` = 0.1, `n_neighbors` = 15, `metric` = cosine.

Based on these characteristics, we hypothesise that HDBSCAN might be best suited for this task, followed by DBSCAN, with k-Medoids likely performing worst due to the non-spherical nature of the projected data. However, empirical evaluation across all three algorithms is necessary to validate these expectations and determine the most effective approach for automatically categorising the Operalog fault descriptions. The subsequent hyperparameter optimisation (Section 3.6) and evaluation will compare their performance quantitatively using the clustering heuristics.

3.5.2 Clustering heuristics

Motivated by the necessity to quantitatively evaluate the quality of different clustering results in this unsupervised setting, three well-understood, internal clustering evaluation metrics (heuristics) are utilised. These metrics assess the ‘goodness-of-fit’ based on the properties of the clusters themselves:

- **Silhouette coefficient** - This metric evaluates how well-separated clusters are by comparing each point’s average distance to points in its own cluster (cohesion) with its average distance to points in the nearest neighbouring cluster (separation) [72].

Scores range from -1 to 1, where values closer to 1 indicate well-separated, dense clusters, values near 0 suggest overlapping clusters, and negative values indicate potential misclassifications.

- **Davies-Bouldin index** - This index quantifies the average ‘similarity’ between each cluster and its most similar neighbour, where similarity is defined as the ratio of within-cluster scatter to between-cluster separation [23]. Lower Davies-Bouldin scores indicate better clustering, implying clusters are compact and well-separated from their nearest neighbours.
- **Calinski-Harabasz index** - Measures the ratio of the sum of between-cluster dispersion to the sum of within-cluster dispersion for all clusters [15]. Higher Calinski-Harabasz scores generally indicate better defined clusters. That is, clusters are dense and well-separated.

Furthermore, each of these metrics captures a different aspect of clustering quality (compactness, separation and variance ratios). As it is unclear which single metric best reflects the desired clustering structure for a specific dataset and task, particularly in an exploratory context like identifying Operalog fault categories, all three metrics were used collectively during the hyperparameter optimisation phase (Section 3.6) to provide a more comprehensive assessment of the different clustering solutions.

3.6 Hyperparameter optimisation

As established in previous sections, achieving effective unsupervised categorisation requires careful selection of parameters for both the dimensionality reduction (UMAP, Section 2.3.2) and clustering stages (Section 2.4). Given the interplay between these stages and the sensitivity of algorithms like UMAP and DBSCAN to their settings, a systematic approach to find optimal parameter combinations is necessary. Furthermore, as discussed in Section 3.5.2 different internal clustering evaluation metrics capture distinct aspects of cluster quality, and it is unclear which single metric best reflects the desired outcome for a specific dataset.

To address these challenges, this project employed the Optuna framework for automatic hyperparameter optimisation [4]. Optuna was selected over alternatives like Hyperband [47] primarily due to its robust support for multi-objective optimisation [4]. This capability is crucial here, as the goal was to simultaneously optimise the clustering results based on multiple evaluation heuristics: the Silhouette coefficient, Davies-Bouldin index, and Calinski-Harabasz index. The optimisation aims to maximise the Silhouette and Calinski-Harabasz scores while minimising the Davies-Bouldin score [72, 23, 15]. A practical advantage of Optuna noted during this work was its ability to save and resume optimisation states, improving research efficiency [4].

An Optuna optimisation session, termed a ‘study’, iteratively evaluates different hyperparameter configurations (‘trials’) [4]. For each trial, Optuna suggests parameter values based on a chosen sampler algorithm. This study utilised the Tree-structured Parzen Estimator sampler, which is Optuna’s default algorithm known for efficiently exploring complex search spaces [97]. The core components of the Optuna study setup were:

- **Objective function** - A function designed to take a set of suggested hyperparameters, execute the relevant parts of the pipeline (UMAP transformation and clustering), calculate the three chosen clustering heuristic scores, and return these scores to Optuna. This function defines the search space by specifying the range or set of possible values for each hyperparameter using Optuna’s library semantics. The specific hyperparameters optimised included shared UMAP settings and algorithm-specific parameters, with ranges informed by preliminary experimentation as shown in Table 3.5. To handle potential errors arising from UMAP’s stochastic nature or problematic parameter combinations leading to clustering failures, the objective function incorporated a retry mechanism (up to 10 attempts). If errors persisted, the trial was assigned extremely poor heuristic scores to guide the optimiser away from that configuration region.
- **Number of trials** - An upper limit defining the maximum number of configurations to evaluate within the study.

Scope	Hyperparameter	Range / Set
General	<code>n_components</code>	Integer range [2, 6]
General	<code>n_neighbors</code>	Integer range [3, 15]
General	<code>min_dist</code>	Float range (0, 0.2)
General	<code>metric</code>	Set { ‘euclidean’, ‘cosine’ }
General	<code>clustering_method</code>	Set { ‘k-Medoids’, ‘DBSCAN’, ‘HDBSCAN’ }
k-Medoids	<code>n_clusters</code>	Integer range [3, 30]
DBSCAN	<code>eps</code>	Float range [0.1, 1.0]
DBSCAN	<code>min_cluster_size</code>	Integer range [3, 12]
HDBSCAN	<code>min_pts</code>	Integer range [3, 30]

Table 3.5: Optuna hyperparameter search space. k-Medoids hyperparameter k was renamed to `n_clusters`; DBSCAN hyperparameter `MinPts` was renamed to `min_cluster_size`; and HDBSCAN hyperparameter m_{pts} was renamed to `min_pts`.

Upon completion of the specified number of trials, the study yields a set of optimal configurations, those for which no single objective (i.e. heuristic) score can be improved without worsening at least one other objective. Since multiple objectives often lead to several such ‘best’ configurations, a post-optimisation filtering step was applied using rank aggregation [48]. For each heuristic, the optimal configurations were ranked, and each configuration received points equal to its inverse rank. For example, for 10 configurations, rank 1 gets 10 points, rank 2 gets 9, ..., rank 10 gets 1 point. These points were summed across all three heuristics for each configuration. The configuration(s) with the highest total score were selected as the final ‘best’ performing set of hyperparameters, representing a balanced choice across the different measures of clustering quality. The quantitative results of this optimisation are presented and discussed in Chapter 4.

3.7 Label generation

After achieving the ‘most optimal’ clusters through the hyperparameter optimisation process described in Section 3.5, the final step involves automatically inferring meaningful names or labels for these discovered groups. This automatic label generation was performed leveraging the natural language processing capabilities of the SpaCy library [32]. The main idea is to capture the essence of the text in each cluster into a precise but representative phrase.

The process begins by examining all the **FaultDescription** sentences assigned to a particular cluster. For each sentence, SpaCy is employed to perform detailed linguistic analysis, including part-of-speech tagging and dependency parsing [86]. This helps identify key components of the grammar and determine their relationships within the sentence.

To focus on the most informative words, several filtering (pre-processing) steps are applied during this analysis. Common English stop-words, along with project-specific stop-words (like ‘ion source’ or ‘breakdown’, as discussed in Section 3.2), are disregarded [86]. Furthermore, tokens that are non-alphanumeric or consist of fewer than two characters are excluded to reduce noise from punctuation or trivial entries.

From the remaining tokens, the system specifically identifies and collects verbs (particularly the root verb of sentences), direct objects (identified via dependency relations), nouns, and adjectives. To ensure consistency and group related word forms, lemmatisation is applied, reducing words to their base or dictionary form. For example, ‘changed’ and ‘changing’ become ‘change’ [86].

Building the label then involves constructing a naive phrase by combining the most frequently occurring elements identified across all sentences in the cluster. Specifically, the system determines:

- The single most common verb (identified as the root of sentences).
- The single most common direct object (based on dependency parsing).
- The top two most common nouns (excluding any words already selected as the verb or direct object to ensure variety).

These selected components are then combined into a single, underscore-separated string to form the cluster’s generated label. This approach incorporates a mechanism to handle cases where fewer than the desired number of unique terms are available within a cluster’s vocabulary, ensuring a label is still generated. The selection process considers a configurable maximum number of top nouns to examine, ensuring efficiency while capturing the primary themes.

Finally, this label generation process is integrated into the data handling workflow. It operates on the clustered data, mapping the generated underscore-separated label back to each original log entry belonging to that cluster.

The intuition behind this heuristic approach is to capture the core semantic meaning of the fault descriptions within a cluster by focusing on the primary action (verb), the entity acted upon (direct object), and the key subjects or items involved (nouns), thereby creating a succinct, machine-generated descriptor for each automatically identified fault category.

3.8 CLI Application

To facilitate the practical application and reproducibility of the methodology developed in this project, a Command Line Interface (CLI) tool was created, accompanying the code submission. This application encapsulates the end-to-end pipeline, providing users with a means to apply the automatic labelling process to Operalog data.

The primary functions offered by the CLI include managing the hyperparameter optimisation process, allowing users to initiate new optimisation runs or resume existing ones, and executing the label inference step on clustered data. Upon completion, the tool generates two key output artefacts: an updated version of the Operalog dataset, augmented with columns indicating the assigned numerical cluster identifier (label) and the inferred natural language label (`real_label`) for each entry, and a structured JSON file. This JSON file contains lists of the original sentences grouped by their assigned cluster, alongside a mapping from each cluster ID to its generated natural language label.

To offer flexibility, the CLI exposes several parameters for user configuration. The primary parameters are listed below:

- **Model** - The HuggingFace model path.
- **Keep punctuation** - Boolean toggle which keeps punctuation from being stripped during the text pre-processing stage.
- **Ignore case** - Boolean toggle which ignores casing during the text pre-processing stage.
- **Clustering methods** - Define one or more clustering methods to use for the hyperparameter optimisation stage.
- **Heuristic methods** - Define one or more heuristic methods to use for the hyperparameter optimisation stage.

The design of the CLI application and its underlying codebase explicitly considers future development. Its structure allows users to experiment with embedding models beyond those researched in this project by simply providing a different model path. Furthermore, the modular organisation of the codebase aims to simplify the future integration of additional clustering algorithms or evaluation heuristics, enhancing the tool’s potential utility for ongoing research or operational use.

Chapter 4

Results and Discussion

This chapter presents the results obtained from the hyperparameter optimisation process detailed in Section 3.6. Recall that the optimisation sought to find the best parameters for the UMAP dimensionality reduction (Section 2.3.2) and subsequent clustering stage (Section 2.4) by evaluating three clustering algorithms (k-Medoids, DBSCAN, HDBSCAN) across both MPNet and Nomic embedding spaces (Figure 3.1). The optimisation used three internal clustering heuristics (Silhouette coefficient, Davies-Bouldin index, Calinski-Harabasz index) as objectives [72, 23, 15], maximising the Silhouette and Calinski-Harabasz scores while minimising the Davies-Bouldin score.

The optimisation was conducted in two main phases:

1. **Head-to-Head (H2H) Phase** - Pairwise comparisons of clustering algorithms, plus an HDBSCAN-only run, were performed to gain initial insights into their relative performance and stability under specific conditions (detailed in Section 4.1.1). These runs used default text pre-processing (ignoring case and ignoring punctuation).
2. **Free-For-All (F4A) Phase** - All three clustering algorithms were compared simultaneously, exploring the impact of different text pre-processing flags (toggling `ignore_case` and `keep_punctuation`) to identify robust, high-performing configurations across various settings (detailed in Section 4.1.2).

For each phase, Optuna [4] studies were run for 400 trials per configuration. This relatively high number of trials was chosen to account for the inherent stochasticity in both the UMAP algorithm (Section 2.3.2) and the Optuna sampler [97], ensuring a more robust exploration of the hyperparameter space defined in Table 3.5. The best configurations from each study were identified using the rank aggregation method described in Section 3.6.

The subjective choice of the final label is based on a qualitative assessment of the labels generated from the quantitatively best-performing cluster configuration identified via rank aggregation. Improving the quantitative evaluation of generated label quality remains an area for future work.

4.1 Experiments and Results

4.1.1 Head-to-Head (H2H) Phase

In the H2H phase, four distinct optimisation studies were conducted for both MPNet and Nomic embeddings, each running for 400 trials, as summarised in Table 4.1. These runs used the default pre-processing settings (`ignore_case = True`, `keep_punctuation = False`).

Run ID	Description	Optimal Config. Plots	Cluster Size Plots
Run A	k-Medoids vs. DBSCAN	Fig. A.10a (MPNet)	Fig. A.12a (MPNet)
		Fig. A.11a (Nomic)	Fig. A.13a (Nomic)
Run B	k-Medoids vs. HDBSCAN	Fig. A.10b (MPNet)	Fig. A.12b (MPNet)
		Fig. A.11b (Nomic)	Fig. A.13b (Nomic)
Run C	DBSCAN vs. HDBSCAN	Fig. A.10c (MPNet)	Fig. A.12c (MPNet)
		Fig. A.11c (Nomic)	Fig. A.13c (Nomic)
Run D	HDBSCAN Only	Fig. A.10d (MPNet)	Fig. A.12d (MPNet)
		Fig. A.11d (Nomic)	Fig. A.13d (Nomic)

Table 4.1: Summary of H2H optimisation runs (default pre-processing). All H2H runs used default pre-processing settings (`ignore_case=True`, `keep_punctuation=False`). Appendix references point to Appendix A.2.

Key performance indicators included the heuristic scores achieved by the top configurations (selected via rank aggregation) and the stability of the clustering results, assessed by examining the distribution of cluster sizes produced by the highlighted top configurations over multiple independent runs. In this case, 10 runs were used here to capture variability stemming from UMAP’s stochasticity. Cluster size distribution provides insight into whether an algorithm consistently finds a similar number of clusters of similar sizes, which is desirable for reproducible categorisation.

Detailed visualisations comparing the heuristic scores for the top-ranked H2H configurations for MPNet and Nomic embeddings are presented in Appendix A.2.2 (Figures A.10 and A.11). These plots show the trade-offs between the different heuristics for the best solutions found. The stability analysis, showing the distribution of cluster sizes over 10 runs for the best configurations identified in the H2H phase, is presented in Appendix A.2.3 (Figures A.12 and A.13).

Observing these distributions (e.g., MPNet results in Figure A.12), several trends emerged:

- DBSCAN often exhibited wide variance in cluster sizes (i.e. a range from 1 to nearly 500 clusters found across runs for optimal configurations in MPNet Run A, Figure A.12a) and sometimes failed to identify competitive optimal configurations (indicated by empty plots in the appendix, i.e. Figure A.12c for MPNet Run C). This suggests higher instability or sensitivity to hyperparameters within this specific data context compared to the other methods. For instance, Nomic Run A (Figure A.13a) configurations using DBSCAN yielded at most 3 clusters, whereas Nomic Run C (Figure A.13c) showed DBSCAN configurations producing up to 40 clusters, highlighting its variable behaviour.
- k-Medoids generally produced stable cluster counts, as expected since the number of

clusters (k) is a hyperparameter optimised within a defined range (Table 3.5). However, in comparisons like Run B (i.e. MPNet, Figure A.10b), k-Medoids configurations were often dominated by HDBSCAN in the optimal set, suggesting HDBSCAN found better heuristic trade-offs.

- HDBSCAN demonstrated reasonable stability in the number and size of clusters identified across runs, particularly when optimised alone (Run D, Figures A.12d and A.13d). In the DBSCAN vs. HDBSCAN comparison (Run C, Figures A.10c and A.11c), HDBSCAN configurations clearly dominated the optimal set, reinforcing its superior performance in this dataset.

These H2H results provide preliminary evidence supporting the initial hypothesis (Section 3.5), that density-based methods capable of handling varying densities (i.e. HDBSCAN) might be better suited to this dataset than the global density-based DBSCAN or the partition-based k-Medoids. Although k-Medoids offers predictable cluster counts, it got dominated by HDBSCAN, indicating that HDBSCAN may be the superior choice on this dataset.

4.1.2 Free-For-All (F4A) Phase

Building on the H2H insights, the F4A phase involved four distinct optimisation studies, each comparing all three clustering methods (k-Medoids, DBSCAN, HDBSCAN) simultaneously for 400 trials per study. These studies explored the impact of the text pre-processing flags (Section 3.2) on the final clustering outcome, as summarised in Table 4.2.

Run ID	ignore_case	keep_punc	Optimal Config. Plots	Cluster Size Plots
Run 1	False	False	Fig. A.14a (MPNet) Fig. A.15a (Nomic)	Fig. A.16a (MPNet) Fig. A.17a (Nomic)
Run 2	False	True	Fig. A.14b (MPNet) Fig. A.15b (Nomic)	Fig. A.16b (MPNet) Fig. A.17b (Nomic)
Run 3 (Default)	True	False	Fig. A.14c (MPNet) Fig. A.15c (Nomic)	Fig. A.16c (MPNet) Fig. A.17c (Nomic)
Run 4	True	True	Fig. A.14d (MPNet) Fig. A.15d (Nomic)	Fig. A.16d (MPNet) Fig. A.17d (Nomic)

Table 4.2: Summary of Free-For-All (F4A) Optimisation Runs. Appendix references point to Appendix A.2. Pre-processor settings `ignore_case` (normalise text casing) and `keep_punc` (keep punctuation) are highlighted. See Section 3.2 for more information.

These runs were performed for both MPNet and Nomic embeddings. The goal was to identify the overall best-performing and most robust configurations considering both the clustering algorithm and the pre-processing variations.

Results from the F4A phase provided insights into the influence of pre-processing choices. Comparing the cluster size distributions in Appendix A.2.5 (Figures A.16 and A.17), Run 2 (`ignore_case=False`, `keep_punc=True`) appeared to perform poorly, in both MPNet and Nomic, where only DBSCAN produced an optimal configuration finding just one clustering (Figure A.16b).

Runs 1 and 3, both featuring `keep_punc=False`, generally exhibited lower variance in cluster sizes compared to Runs 2 and 4 (`keep_punc=True`), especially visible in the DBSCAN

results (known for variability from Section 4.1.1). This suggests that removing punctuation (`keep_punc=False`) is beneficial, likely because punctuation tokens introduced noise which was detrimental to the embedding models’ semantic representations for this dataset. Comparing Run 1 (`ignore_case=False`) and Run 3 (`ignore_case=True`), both with punctuation removed, Run 3 (ignoring case) often showed slightly more consistent results (especially in the Nomic embeddings), suggesting that ignoring case standardisation also helps.

However, HDBSCAN configurations still produced some outliers in terms of cluster size across most runs, indicating sensitivity in certain parameter regions. However, these outliers may be more useful configurations, as HDBSCAN tends to produce very conservative configurations with low cluster sizes of around 3 to 5.

The extreme outlier cluster size (>400) found in MPNet Run 1 (Figure A.16a) might highlight a specific sensitivity or edge case within the optimisation process for that particular setting. Overall, the F4A results reinforce the preference for HDBSCAN and suggest that the default pre-processing (Run 3: `ignore_case=True`, `keep_punc=False`) provides a good balance of performance and stability.

4.2 Distribution of Hyperparameters

To gain further insight into the parameter settings favoured by the optimisation process, this section examines the distribution of hyperparameters corresponding to the optimal configurations identified by Optuna across the H2H and F4A phases. Appendix A.2.1 presents visualisations of these distributions, referencing the hyperparameters defined in Table 3.5. Specifically, it shows the aggregated distributions for:

- All optimal configurations found by Optuna before rank aggregation.
- The top 5 configurations selected from each study via rank aggregation (as detailed in Section 3.6).

These distributions are provided separately for the combined results (‘Overall’), for runs using MPNet embeddings, and for runs using Nomic embeddings, as summarised in Table 4.3.

Aggregation Type	Number of Config.	Optimal Config. Plots	Top 5 Plots
Overall	288	Fig. A.7a	Fig. A.7b
MPNet	144	Fig. A.8a	Fig. A.8b
Nomic	144	Fig. A.9a	Fig. A.9b

Table 4.3: Summary of Optuna optimal hyperparameter plots. The top 5 hyperparameter plots are selected via rank aggregation. Appendix references point to Appendix A.2. Parameters correspond to Table 3.5 See Section 3.6 for more information.

Analysis of the hyperparameter distributions (Figures A.7 to A.9) reveals several notable trends among the optimal configurations:

- **Distance Metric** - Confirming preliminary observations (Section 3.4.2), the ‘euclidean’ distance metric was frequently preferred over ‘cosine’, particularly for Nomic

embeddings. Over 60% of the overall top 5 configurations utilised ‘euclidean’ distance, rising to nearly 80% among the top 5 configurations specifically for Nomic (Figure A.9b). This contradicts the common heuristic favouring cosine distance for text embeddings [8, 17].

- **UMAP Components** - Optimal solutions consistently favoured a higher number of UMAP components, typically 5 or 6 (the upper end of the search range), suggesting that retaining slightly higher dimensionality post-UMAP aids clustering performance for this dataset.
- **Clustering Algorithm Preference** - The preferred clustering algorithm differed between embedding models in the top 5 configurations. MPNet runs frequently favoured DBSCAN (over 40%), whereas Nomic runs showed a preference for HDBSCAN (around 40%) (Figures A.8b and A.9b).
- **Density Parameters (MinPts/m_{pts})** - For DBSCAN, MPNet’s optimal runs often selected very low MinPts values, which might contribute to the observed instability and poorer performance of DBSCAN with MPNet embeddings. Conversely, Nomic’s optimal DBSCAN runs preferred MinPts values around 7-8. For HDBSCAN, preferred m_{pts} values varied but often fell within the lower to mid-range of the search space [3, 30].
- **k-Medoids Clusters** - Ignoring outliers, the top 5 k-Medoids configurations tended to favour cluster counts (k) in the range of 18 to 21, aligning reasonably well with the broader 5-25 range estimated from HDBSCAN stability.
- **UMAP min_dist** - The distribution for min_dist was heavily left-skewed towards lower values (closer to 1) across both embeddings and both optimal sets (all vs. top 5). This strong preference suggests the defined search space (0,0.2) might have been slightly too broad, as the optimizer rarely favoured values near the upper bound.

4.3 Discussion

The analysis of cluster sizes across the H2H and F4A experiments (Appendices A.2.3 and A.2.5) suggests a plausible range for the number of distinct fault categories within the ion source data. Excluding extreme outliers and configurations resulting in very few clusters (often associated with DBSCAN instability or less optimal methods), the more stable HDBSCAN configurations frequently identified between approximately 5 and 25 clusters. This range may represent a reasonable estimate for the number of operationally relevant fault categories inferable from this dataset using the current methodology, although further validation is required.

The experimental results, combining insights from the H2H and F4A phases and the hyperparameter distribution analysis (Section 4.2), highlight several key aspects of applying unsupervised clustering to the ISIS Operalog data:

1. **DBSCAN Instability** - The observed instability of DBSCAN, particularly its variance in cluster counts and occasional failure to find competitive solutions, suggests its

reliance on a single global density threshold (`Eps`) is a limitation for the Operalog’s likely varying cluster densities. The preference for very low `MinPts` in optimal MPNet runs might exacerbate this issue.

2. **HDBSCAN Robustness** - HDBSCAN emerged as a more robust density-based method, consistently finding ‘good’ solutions and demonstrating better stability. Its ability to handle varying densities [16] appears advantageous, aligning with its preference among top configurations for Nomic embeddings.
3. **k-Medoids Suitability** - While providing stable cluster counts (often favouring 18-21 clusters in top runs), k-Medoids’ appropriateness remains questionable given the non-spherical cluster shapes suggested by UMAP projections (Figure 3.3). The optimal heuristic scores might not perfectly reflect meaningful operational categories if the shape assumption is violated.
4. **Pre-processing Impact** - The F4A phase confirmed that removing punctuation (`keep_punc=False`) and ignoring case (`ignore_case=True`) generally yielded more stable and quantitatively better clustering results, likely by reducing noise in the embeddings.
5. **Embedding Model Nuances** - While neither MPNet nor Nomic showed dramatic overall superiority, Nomic embeddings potentially aligned more frequently with the inferred 5-25 cluster range and yielded fewer extreme outliers with HDBSCAN. Furthermore, the analysis revealed distinct preferences - Nomic strongly favoured the ‘euclidean’ distance metric and HDBSCAN clustering in its top configurations, whereas MPNet showed a greater tendency towards the ‘cosine’ metric and DBSCAN.
6. **UMAP Parameter Insights** - Optimal configurations consistently preferred higher dimensionality (5-6 components) after UMAP reduction and strongly favoured lower `min_dist` values, suggesting denser packing in the reduced space aids the clustering algorithms used. The preference for the ‘euclidean’ metric over ‘cosine’, especially for Nomic, is a noteworthy finding that warrants further investigation, potentially indicating specific characteristics of the Nomic embedding space or the UMAP implementation.
7. **Heuristic-Based Optimisation** - The process confirmed the value of multi-objective optimisation using several heuristics, as configurations balanced trade-offs differently. However, the ultimate validation requires assessing the semantic and operational utility of the resulting categories and labels, representing a crucial area for future work. The potential need for more domain-specific or task-aware evaluation metrics beyond standard internal heuristics could also be considered.

Finally, selecting the single definitive configuration for final label generation presents challenges. While Optuna and rank aggregation identify quantitatively ‘optimal’ configurations, choosing among the top set requires balancing heuristic scores, stability (cluster counts), parameter plausibility (i.e. reasonable `MinPts`), and ultimately, the interpretability of the resulting clusters. The inherent stochasticity within the pipeline (UMAP) adds complexity, reinforcing that validation relies on qualitative assessment and operational utility. Therefore,

refining selection criteria and developing methods for quantitatively assessing label quality are important future directions.

Chapter 5

Conclusion

This project successfully developed and evaluated an end-to-end unsupervised machine learning pipeline for the automatic categorisation of unstructured operational log entries from the ISIS Neutron and Muon Source’s Operalog, focusing specifically on `FaultDescription` data related to the ion source equipment. Recognising the open-ended nature of this task and the lack of established industry benchmarks. The primary contribution lies in demonstrating the feasibility of applying modern NLP techniques, specifically sentence embeddings combined with dimensionality reduction and clustering, to extract meaningful structure from complex, domain-specific textual data.

The core methodology involved pre-processing the text, generating embeddings using MPNet [79] and Nomic [62] models, reducing dimensionality with UMAP [53], performing clustering using k-Medoids [38], DBSCAN [26], and HDBSCAN [16], and systematically optimising hyperparameters via Optuna [4] based on multiple internal clustering heuristics [72, 23, 15]. The experimental results presented in Chapter 4 and Appendix A.2 provided valuable insights. Notably, HDBSCAN [16] consistently emerged as a highly applicable clustering algorithm, effectively handling varying cluster densities and demonstrating superior stability compared to DBSCAN and k-Medoids. The hyperparameter analysis revealed a preference for the ‘euclidean’ distance metric in UMAP, particularly with Nomic embeddings, and indicated that retaining slightly higher dimensionality (5-6 components) post-reduction aids clustering. Furthermore, optimal pre-processing involved removing punctuation and ignoring text case. While both MPNet and Nomic embeddings enabled effective clustering, Nomic showed potential advantages, aligning more frequently with the inferred range of 5-25 distinct categories and exhibiting fewer extreme outliers when paired with HDBSCAN, which was its preferred algorithm in top configurations (unlike MPNet which favoured DBSCAN, shown to be unstable). A naive, heuristic-based method for automatically generating cluster labels using SpaCy [32] was implemented, and the entire process was encapsulated within a reusable CLI tool (Section 3.8) to facilitate further experimentation.

Despite the successful demonstration of the pipeline’s capability, several challenges and limitations remain, as discussed in Section 4.3. Selecting the single best configuration from the set of optimal solutions produced by multi-objective optimisation is non-trivial, requiring careful consideration of the trade-offs between quantitative heuristics, parameter plausibility,

and qualitative assessment of cluster interpretability. The inherent stochasticity, particularly from UMAP, also adds complexity to achieving perfectly reproducible cluster assignments. Furthermore, the current label generation method requires significant refinement to ensure the labels are truly informative and operationally useful.

These limitations naturally lead to several avenues for future work:

1. **Fine-tuning PLMs** - The performance of the pipeline could potentially be improved by fine-tuning the embedding models (MPNet or Nomic) specifically on the Operalog data or related technical documentation from ISIS. This could adapt the models more closely to the domain-specific language and nuances, potentially leading to more discriminative embeddings and possibly reconciling differing algorithmic preferences.
2. **Exploring Alternative Models & Metrics** - Given the rapid pace of development in NLP, investigating newer embedding models could yield benefits, perhaps including those specifically optimised for Euclidean space if this metric preference persists. However, this may necessitate access to greater computational resources. Further investigation into the unexpected preference for ‘euclidean’ over ‘cosine’ distance is also warranted.
3. **Hyperparameter Search Refinement** - The hyperparameter search space could be further refined based on the observed distributions, for example, by narrowing the range for UMAP’s `min_dist` or exploring different ranges for density parameters like `MinPts`.
4. **Refined Label Generation** - The current naive label generation (Section 3.7) is a key area for improvement. More sophisticated techniques, such as leveraging topic modelling [20] within clusters or using abstractive summarization models [60], should be explored. Evaluating label quality quantitatively also remains an open challenge.
5. **Broader Data Scope** - The analysis should be extended beyond the ‘ion source’ equipment type to encompass the full range of systems documented in the Operalog, assessing the pipeline’s generalisability.
6. **Integration with Structured Data** - Combining the insights from this text-based analysis with findings from the partner project focusing on structured ion source logs could yield a more holistic understanding of fault patterns and potentially improve predictive maintenance models.

Finally, it is crucial to consider the ethical implications inherent in using complex machine learning models, particularly PLMs derived from external sources. As discussed in Section 2.2.1, relying on pre-trained models introduces a dependency on the practices of the model creators [99]. There are potential risks associated with models trained on vast, often opaque datasets, including the possibility of inheriting societal biases present in the training data, or potential data contamination issues that might affect model behaviour in unforeseen ways [56]. While less likely with reputable sources, the theoretical risk of maliciously manipulated models also exists. Therefore, a commitment to using models from transparent sources,

critically evaluating their outputs, and understanding their limitations is paramount. Furthermore, handling operational data like the Operalog requires adherence to appropriate data privacy and confidentiality protocols, ensuring sensitive information is protected throughout the analysis pipeline.

In conclusion, this project has successfully demonstrated the potential of applying an unsupervised NLP pipeline, leveraging state-of-the-art embedding and clustering techniques, to automatically categorise unstructured operational logs from the complex ISIS facility. While further refinements, particularly in label generation and configuration selection, are necessary for direct operational deployment, the developed methodology, incorporating detailed hyperparameter analysis and encapsulated in a CLI tool, provides a valuable foundation. This work represents significant progress towards unlocking the rich information contained within textual logs, paving the way for improved diagnostics, data-driven insights, and ultimately, more effective predictive maintenance strategies at large-scale scientific facilities.

Bibliography

- [1] Martín Abadi, Ashish Agarwal, Paul Barham, Eugene Brevdo, Zhifeng Chen, Craig Citro, Greg S. Corrado, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Ian Goodfellow, Andrew Harp, Geoffrey Irving, Michael Isard, Yangqing Jia, Rafal Jozefowicz, Lukasz Kaiser, Manjunath Kudlur, Josh Levenberg, Dandelion Mané, Rajat Monga, Sherry Moore, Derek Murray, Chris Olah, Mike Schuster, Jonathon Shlens, Benoit Steiner, Ilya Sutskever, Kunal Talwar, Paul Tucker, Vincent Vanhoucke, Vijay Vasudevan, Fernanda Viégas, Oriol Vinyals, Pete Warden, Martin Wattenberg, Martin Wicke, Yuan Yu, and Xiaoqiang Zheng. TensorFlow: Large-scale machine learning on heterogeneous systems, 2015. URL <https://www.tensorflow.org/>. Software available from tensorflow.org.
- [2] Hervé Abdi and Lynne J Williams. Principal component analysis. *Wiley interdisciplinary reviews: computational statistics*, 2(4):433–459, 2010.
- [3] PY Abijith, Piyush Patidar, Gaurav Nair, and Rohan Pandya. Large language models trained on equipment maintenance text. In *Abu Dhabi International Petroleum Exhibition and Conference*, page D021S065R003. SPE, 2023.
- [4] Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining*, pages 2623–2631, 2019.
- [5] Ramiz M Aliguliyev. A new sentence similarity measure and sentence based extractive technique for automatic text summarization. *Expert Systems with Applications*, 36(4):7764–7772, 2009.
- [6] C Ankenbrandt, C Curtis, C Hojvat, RP Johnson, C Owen, C Schmidt, L Teng, and RC Webber. H- charge exchange injection systems. In *11th International Conference on High-Energy Accelerators: Geneva, Switzerland, July 7–11, 1980*, pages 260–271. Springer, 1980.
- [7] Asim Yaqoob. First line diagnosis at ISIS, Oct 2017. URL <https://indico.cern.ch/event/558933/contributions/2724364>. Last visited 2025-04-15.
- [8] Kensuke Baba, Tetsuya Nakatoh, and Toshiro Minami. Plagiarism detection using document similarity based on distributed representation. *Procedia computer science*, 111:382–387, 2017.
- [9] Ioana Baldini, Stephen J Fink, and Erik Altman. Predicting gpu performance from cpu runs using machine learning. In *2014 IEEE 26th International Symposium on Computer Architecture and High Performance Computing*, pages 254–261. IEEE, 2014.
- [10] Steven Bird, Ewan Klein, and Edward Loper. *Natural language processing with Python: analyzing text with the natural language toolkit*. " O'Reilly Media, Inc.", 2009.
- [11] Piotr Bojanowski, Edouard Grave, Armand Joulin, and Tomas Mikolov. Enriching word vectors with subword information. *Transactions of the association for computational linguistics*, 5:135–146, 2017.
- [12] Tawfik Borgi, Adel Hidri, Benjamin Neef, and Mohamed Saber Naceur. Data analytics for predictive maintenance of industrial robots. In *2017 International Conference on Advanced Systems and Electric Technologies (IC_ASET)*, pages 412–417. IEEE, 2017.
- [13] Alexander Budanitsky and Graeme Hirst. Semantic distance in wordnet: An experimental, application-oriented evaluation of

- five measures. In *Workshop on Word-Net and other lexical resources*, volume 2, pages 2–2, 2001.
- [14] T Tony Cai and Rong Ma. Theoretical foundations of t-sne for visualizing high-dimensional clustered data. *Journal of Machine Learning Research*, 23(301):1–54, 2022.
- [15] Tadeusz Caliński and Jerzy Harabasz. A dendrite method for cluster analysis. *Communications in Statistics-theory and Methods*, 3(1):1–27, 1974.
- [16] Ricardo JGB Campello, Davoud Moulavi, and Jörg Sander. Density-based clustering based on hierarchical density estimates. In *Pacific-Asia conference on knowledge discovery and data mining*, pages 160–172. Springer, 2013.
- [17] Hongliu Cao. Recent advances in text embedding: A comprehensive review of top-performing methods on the mteb benchmark. *arXiv preprint arXiv:2406.01607*, 2024.
- [18] Thyago P Carvalho, Fabrizzio AAMN Soares, Roberto Vita, Roberto da P Francisco, João P Basto, and Symone GS Alcalá. A systematic literature review of machine learning methods applied to predictive maintenance. *Computers & Industrial Engineering*, 137:106024, 2019.
- [19] Coenen, Andy and Pearce, Adam. Understanding UMAP, 2020. URL <https://pair-code.github.io/understanding-umap/>. Last visited 2025-04-21.
- [20] Stephan A Curiskis, Barry Drake, Thomas R Osborn, and Paul J Kennedy. An evaluation of document clustering and topic modelling in two online social networks: Twitter and reddit. *Information Processing & Management*, 57(2):102034, 2020.
- [21] Zihang Dai, Zhilin Yang, Yiming Yang, Jaime Carbonell, Quoc V Le, and Ruslan Salakhutdinov. Transformer-xl: Attentive language models beyond a fixed-length context. *arXiv preprint arXiv:1901.02860*, 2019.
- [22] Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. Flashattention: Fast and memory-efficient exact attention with io-awareness. *Advances in neural information processing systems*, 35:16344–16359, 2022.
- [23] David L Davies and Donald W Bouldin. A cluster separation measure. *IEEE transactions on pattern analysis and machine intelligence*, (2):224–227, 1979.
- [24] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*, pages 4171–4186, 2019.
- [25] Sergey Edunov, Alexei Baevski, and Michael Auli. Pre-trained language model representations for language generation. *arXiv preprint arXiv:1903.09722*, 2019.
- [26] Martin Ester, Hans-Peter Kriegel, Jörg Sander, Xiaowei Xu, et al. A density-based algorithm for discovering clusters in large spatial databases with noise. In *kdd*, volume 96, pages 226–231, 1996.
- [27] Maurizio Faccio, Alessandro Persona, Fabio Sgarbossa, and Giorgia Zanin. Industrial maintenance policy development: A quantitative framework. *International Journal of Production Economics*, 147: 85–93, 2014.
- [28] Bolin Gao and Lacra Pavel. On the properties of the softmax function with application in game theory and reinforcement learning. *arXiv preprint arXiv:1704.00805*, 2017.
- [29] Gregory Grefenstette. Tokenization. In *Syntactic wordclass tagging*, pages 117–133. Springer, 1999.
- [30] John M Harris. *Combinatorics and graph theory*. Springer, 2008.
- [31] Alexander Hinneburg and Daniel A Keim. Optimal grid-clustering: Towards breaking the curse of dimensionality in high-dimensional clustering. 1999.

- [32] Matthew Honnibal and Ines Montani. spaCy 2: Natural language understanding with Bloom embeddings, convolutional neural networks and incremental parsing. 2017.
- [33] Harold Hotelling. Analysis of a complex of statistical variables into principal components. *Journal of educational psychology*, 24(6):417, 1933.
- [34] ISIS Neutron and Muon Source. A Practical Guide to the ISIS neutron and Muon source, Apr 2021. URL https://www.isis.stfc.ac.uk/Pages/News21_PracticalGuide.aspx. Last visited 2025-04-15.
- [35] ISIS Neutron and Muon Source. ISIS beam operations, Apr 2024. URL <https://www.isis.stfc.ac.uk/Pages/beam-status.aspx>. Last visited 2025-04-15.
- [36] Ali Jezzini, Mohammad Ayache, Lina Elkhansa, Bassem Makki, and Maya Zein. Effects of predictive maintenance (pdm), proactive maintenace (pom) & preventive maintenance (pm) on minimizing the faults in medical instruments. In *2013 2nd International conference on advances in biomedical engineering*, pages 53–56. IEEE, 2013.
- [37] Taeho Jo. K nearest neighbor for text summarization using feature similarity. In *2017 International Conference on Communication, Control, Computing and Electronics Engineering (ICCC-CEE)*, pages 1–5. IEEE, 2017.
- [38] Leonard Kaufman and Peter Rousseeuw, J. *Partitioning Around Medoids (Program PAM)*, chapter 2, pages 68–125. John Wiley and Sons, Ltd, 1990.
- [39] Leonard Kaufman and Peter J Rousseeuw. *Finding groups in data: an introduction to cluster analysis*. John Wiley & Sons, 2009.
- [40] Sushil Kumar and Komal Kumar Bhatia. Semantic similarity and text summarization based novelty detection. *SN Applied Sciences*, 2(3):332, 2020.
- [41] Aditya Kusupati, Gantavya Bhatt, Aniket Rege, Matthew Wallingford, Aditya Sinha, Vivek Ramanujan, William Howard-Snyder, Kaifeng Chen, Sham Kakade, Prateek Jain, et al. Matryoshka representation learning. *Advances in Neural Information Processing Systems*, 35:30233–30249, 2022.
- [42] Thomas K Landauer, Peter W Foltz, and Darrell Laham. An introduction to latent semantic analysis. *Discourse processes*, 25(2-3):259–284, 1998.
- [43] Jinhyuk Lee, Zhuyun Dai, Xiaoqi Ren, Blair Chen, Daniel Cer, Jeremy R Cole, Kai Hui, Michael Boratko, Rajvi Kapadia, Wen Ding, et al. Gecko: Versatile text embeddings distilled from large language models. *arXiv preprint arXiv:2403.20327*, 2024.
- [44] John M Lee. *Smooth manifolds*. Springer, 2003.
- [45] John M Lee. *Riemannian manifolds: an introduction to curvature*, volume 176. Springer Science & Business Media, 2006.
- [46] Hongfei Li, Dhaivat Parikh, Qing He, Buyue Qian, Zhiguo Li, Dongping Fang, and Arun Hampapur. Improving rail network velocity: A machine learning approach to predictive maintenance. *Transportation Research Part C: Emerging Technologies*, 45:17–26, 2014.
- [47] Lisha Li, Kevin Jamieson, Giulia DeSalvo, Afshin Rostamizadeh, and Ameet Talwalkar. Hyperband: A novel bandit-based approach to hyperparameter optimization. *Journal of Machine Learning Research*, 18(185):1–52, 2018.
- [48] Xue Li, Xinlei Wang, and Guanghua Xiao. A comparative study of rank aggregation methods for partial and top ranked lists in genomic applications. *Briefings in bioinformatics*, 20(1):178–189, 2019.
- [49] Hong Liang, Xiao Sun, Yunlei Sun, and Yuan Gao. Text feature extraction based on deep learning: a review. *EURASIP journal on wireless communications and networking*, 2017:1–12, 2017.
- [50] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke

- Zettlemoyer, and Veselin Stoyanov. Roberta: A robustly optimized bert pretraining approach. *arXiv preprint arXiv:1907.11692*, 2019.
- [51] Romans Lukashenko, Vita Graudina, and Janis Grundspenkis. Computer-based plagiarism detection methods and tools: an overview. In *Proceedings of the 2007 international conference on Computer systems and technologies*, pages 1–6, 2007.
- [52] James MacQueen. Some methods for classification and analysis of multivariate observations. In *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability, Volume 1: Statistics*, volume 5, pages 281–298. University of California press, 1967.
- [53] Leland McInnes, John Healy, and James Melville. Umap: Uniform manifold approximation and projection for dimension reduction. *arXiv preprint arXiv:1802.03426*, 2018.
- [54] HT Michael. Electronic circuits: fundamentals and applications, 2006.
- [55] Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. Efficient estimation of word representations in vector space. *arXiv preprint arXiv:1301.3781*, 2013.
- [56] Bonan Min, Hayley Ross, Elior Sulem, Amir Pouran Ben Veyseh, Thien Huu Nguyen, Oscar Sainz, Eneko Agirre, Ilana Heintz, and Dan Roth. Recent advances in natural language processing via large pre-trained language models: A survey. *ACM Computing Surveys*, 56(2):1–40, 2023.
- [57] R Keith Mobley. *An introduction to predictive maintenance*. Elsevier, 2002.
- [58] Niklas Muennighoff, Nouamane Tazi, Loïc Magne, and Nils Reimers. Mteb: Massive text embedding benchmark. *arXiv preprint arXiv:2210.07316*, 2022.
- [59] Dietrich W Müller and Günther Sawitzki. Excess mass estimates and tests for multimodality. *Journal of the American Statistical Association*, 86(415):738–746, 1991.
- [60] Ramesh Nallapati, Bowen Zhou, Caglar Gulcehre, Bing Xiang, et al. Abstractive text summarization using sequence-to-sequence rnns and beyond. *arXiv preprint arXiv:1602.06023*, 2016.
- [61] Giancarlo Nota, Alberto Postiglione, and Rosario Carvello. Text mining techniques for the management of predictive maintenance. *Procedia Computer Science*, 200: 778–792, 2022.
- [62] Zach Nussbaum, John X Morris, Brandon Duderstadt, and Andriy Mulyar. Nomic embed: Training a reproducible long context text embedder. *arXiv preprint arXiv:2402.01613*, 2024.
- [63] Nussbaum, Zach and Cembalest, Max. Unboxing Nomic Embed v1.5: Resizable Production Embeddings with Matryoshka Representation Learning, Feb 2024. URL <https://www.nomic.ai/blog/posts/nomic-embed-matryoshka>. Last visited 2025-04-20.
- [64] Nussbaum, Zach and Cembalest, Max. Nomic Embed’s Surprisingly Good MTEB Arena Elo Score, Aug 2024. URL <https://www.nomic.ai/blog/posts/evaluating-embedding-models>. Last visited 2025-04-18.
- [65] Layla Oesper, Daniele Merico, Ruth Isserlin, and Gary D Bader. Wordcloud: a cytoscape plugin to create a visual semantic summary of networks. *Source code for biology and medicine*, 6(1):7, 2011.
- [66] Zhaotai Pan, Yi Ge, Yu Chen Zhou, Jing Chang Huang, Yu Ling Zheng, Ning Zhang, Xiao Xing Liang, Peng Gao, Guan Qun Zhang, Qingyan Wang, et al. Cognitive acoustic analytics service for internet of things. In *2017 IEEE International Conference on Cognitive Computing (ICCC)*, pages 96–103. IEEE, 2017.
- [67] Karl Pearson. Liii. on lines and planes of closest fit to systems of points in space. *The London, Edinburgh, and Dublin philosophical magazine and journal of science*, 2(11):559–572, 1901.
- [68] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel,

- M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.
- [69] Jeffrey Pennington, Richard Socher, and Christopher D Manning. Glove: Global vectors for word representation. In *Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP)*, pages 1532–1543, 2014.
- [70] Alec Radford, Karthik Narasimhan, Tim Salimans, Ilya Sutskever, et al. Improving language understanding by generative pre-training. 2018.
- [71] T Nora Raju, PA Rahana, Raichel Moncy, Sreedarsana Ajay, and Sindhya K Nambiar. Sentence similarity-a state of art approaches. In *2022 International Conference on Computing, Communication, Security and Intelligent Systems (IC3SIS)*, pages 1–6. IEEE, 2022.
- [72] Peter J Rousseeuw. Silhouettes: a graphical aid to the interpretation and validation of cluster analysis. *Journal of computational and applied mathematics*, 20: 53–65, 1987.
- [73] William Rudman, Nate Gillman, Taylor Rayne, and Carsten Eickhoff. Isoscore: Measuring the uniformity of embedding space utilization. *arXiv preprint arXiv:2108.07344*, 2021.
- [74] Mark Sammons, Christos Christodoulopoulos, Parisa Kordjamshidi, Daniel Khashabi, Vivek Srikumar, and Dan Roth. Edison: Feature extraction for nlp, simplified. In *Proceedings of the Tenth International Conference on Language Resources and Evaluation (LREC’16)*, pages 4085–4092, 2016.
- [75] Justyna Sarzynska-Wawer, Aleksander Wawer, Aleksandra Pawlak, Julia Szymanowska, Izabela Stefaniak, Michal Jarkiewicz, and Lukasz Okruszek. Detecting formal thought disorder by deep contextualized word representations. *Psychiatry Research*, 304:114135, 2021.
- [76] Roger C Schank. Conceptual dependency: A theory of natural language understanding. *Cognitive psychology*, 3(4):552–631, 1972.
- [77] BP Sharma. Nuclear reactors: Moderator and reflector materials. *Encyclopedia of Materials: Science and Technology*, pages 6365–6369, 2001.
- [78] Catarina Silva and Bernardete Ribeiro. The importance of stop word removal on recall values in text categorization. In *Proceedings of the International Joint Conference on Neural Networks, 2003.*, volume 3, pages 1661–1666. IEEE, 2003.
- [79] Kaitao Song, Xu Tan, Tao Qin, Jianfeng Lu, and Tie-Yan Liu. Mpnet: Masked and permuted pre-training for language understanding. *Advances in neural information processing systems*, 33:16857–16867, 2020.
- [80] Carlos Oscar Sánchez Sorzano, Javier Vargas, and A Pascual Montano. A survey of dimensionality reduction techniques. *arXiv preprint arXiv:1403.2877*, 2014.
- [81] Josef Steinberger and Karel Ježek. Text summarization and singular value decomposition. In *Advances in Information Systems: Third International Conference, ADVIS 2004, Izmir, Turkey, October 20-22, 2004. Proceedings 3*, pages 245–254. Springer, 2005.
- [82] Chuan-Jun Su and Shi-Feng Huang. Real-time big data analytics for hard disk drive predictive maintenance. *Computers & Electrical Engineering*, 71:93–101, 2018.
- [83] Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024.
- [84] Gian Antonio Susto and Alessandro Beghi. Dealing with time-series data in predictive maintenance problems. In *2016 IEEE 21st International Conference on Emerging Technologies and Factory Automation (ETFA)*, pages 1–4. IEEE, 2016.

- [85] Gian Antonio Susto, Alessandro Beghi, and Cristina De Luca. A predictive maintenance system for epitaxy processes based on filtering and prediction techniques. *IEEE Transactions on Semiconductor Manufacturing*, 25(4):638–649, 2012.
- [86] Ayisha Tabassum and Rajendra R Patil. A survey on text pre-processing & feature extraction techniques in natural language processing. *International Research Journal of Engineering and Technology (IR-JET)*, 7(06):4864–4867, 2020.
- [87] Wilson L Taylor. “cloze procedure”: A new tool for measuring readability. *Journalism quarterly*, 30(4):415–433, 1953.
- [88] JWG Thomason. The isis spallation neutron and muon source—the first thirty-three years. *Nuclear Instruments and Methods in Physics Research Section A: Accelerators, Spectrometers, Detectors and Associated Equipment*, 917:61–67, 2019.
- [89] Madeleine Udell, Corinne Horn, Reza Zadeh, Stephen Boyd, et al. Generalized low rank models. *Foundations and Trends® in Machine Learning*, 9(1):1–118, 2016.
- [90] Juan Pablo Usuga-Cadavid, Samir Lamouri, Bernard Grabot, and Arnaud Fortin. Using deep learning to value free-form text data for predictive maintenance. *International Journal of Production Research*, 60(14):4548–4575, 2022.
- [91] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.
- [92] Michel Verleysen and Damien François. The curse of dimensionality in data mining and time series prediction. In *International work-conference on artificial neural networks*, pages 758–770. Springer, 2005.
- [93] Geert Waeyenbergh and Liliane Pintelon. A framework for maintenance concept development. *International journal of production economics*, 77(3):299–313, 2002.
- [94] Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel R Bowman. Glue: A multi-task benchmark and analysis platform for natural language understanding. *arXiv preprint arXiv:1804.07461*, 2018.
- [95] Liang Wang, Nan Yang, Xiaolong Huang, Linjun Yang, Rangan Majumder, and Furu Wei. Improving text embeddings with large language models. *arXiv preprint arXiv:2401.00368*, 2023.
- [96] Liang Wang, Nan Yang, Xiaolong Huang, Linjun Yang, Rangan Majumder, and Furu Wei. Multilingual e5 text embeddings: A technical report. *arXiv preprint arXiv:2402.05672*, 2024.
- [97] Shuhei Watanabe. Tree-structured parzen estimator: Understanding its algorithm components and their roles for better empirical performance. *arXiv preprint arXiv:2304.11127*, 2023.
- [98] Stephen Willard. *General topology*. Courier Corporation, 2012.
- [99] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, et al. Huggingface’s transformers: State-of-the-art natural language processing. *arXiv preprint arXiv:1910.03771*, 2019.
- [100] An Yang, Baosong Yang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Zhou, Chengpeng Li, C Li, D Liu, F Huang, et al. Qwen2 technical report. arxiv 2024. *arXiv preprint arXiv:2407.10671*, 2024.
- [101] Zhilin Yang, Zihang Dai, Yiming Yang, Jaime Carbonell, Russ R Salakhutdinov, and Quoc V Le. Xlnet: Generalized autoregressive pretraining for language understanding. *Advances in neural information processing systems*, 32, 2019.
- [102] Minghe Yu, Guoliang Li, Dong Deng, and Jianhua Feng. String similarity search and join: a survey. *Frontiers of Computer Science*, 10:399–417, 2016.

- [103] Zhe Zhang, Junxi Zhang, and Huifeng Xue. Improved k-means clustering algorithm. In *2008 Congress on image and signal processing*, volume 5, pages 169–172. IEEE, 2008.
- [104] Zhilu Zhang and Mert Sabuncu. Generalized cross entropy loss for training deep neural networks with noisy labels. *Advances in neural information processing systems*, 31, 2018.

Appendix A

Results and Figures

A.1 UMAP Hyperparameter Search

Figures [A.1](#) through [A.6](#) in this appendix illustrate the results of the naive grid search performed over the UMAP hyperparameters listed in [Table 3.4](#), applied to both MPNet and Nomic embeddings of the Operalog `FaultDescription` entries. These visualisations demonstrate the effects of varying parameters like `n_neighbors`, `min_dist` and `metric` as described conceptually in [Section 2.3.2](#). Notably, visual inspection suggests that, for the goal of creating a low-dimensional representation that isolates structural groupings in this specific dataset, the ‘cosine’ distance metric did not produce significantly different or clearly superior results compared to the ‘euclidean’ metric. This outcome is somewhat surprising as cosine distance is frequently recommended for text embedding applications in the literature [\[17, 8\]](#). This observation, combined with the difficulty in determining the optimal settings purely through visual inspection of numerous plots, makes it evident that this naive grid search approach is insufficient. It therefore motivates the need for the systematic hyperparameter optimisation process detailed in [Section 3.6](#).



Figure A.1: MPNet, grid search for UMAP hyperparameter (see Table 3.4) over the parameters `metric` and `min_dist`

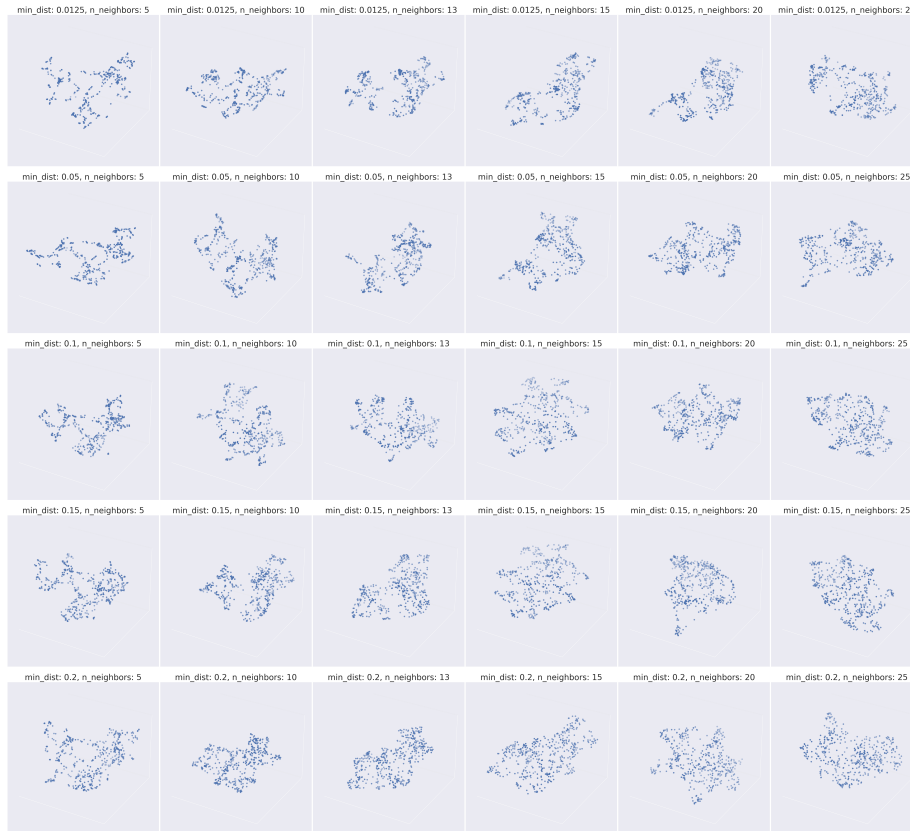


Figure A.2: MPNet, grid search for UMAP hyperparameter (see Table 3.4) over the parameters `min_dist` and `n_neighbors`



Figure A.3: MPNet, grid search for UMAP hyperparameter (see Table 3.4) over the parameters `metric` and `n_neighbors`



Figure A.4: Nomic, grid search for UMAP hyperparameter (see Table 3.4) over the parameters `metric` and `min_dist`



Figure A.5: Nomic, grid search for UMAP hyperparameter (see Table 3.4) over the parameters `min_dist` and `n_neighbors`



Figure A.6: Nomic, grid search for UMAP hyperparameter (see Table 3.4) over the parameters `metric` and `n_neighbors`

A.2 Experimental Results Plots

This appendix section presents supplementary visualisations supporting the hyperparameter optimisation results discussed in Chapter 4. The plots detail outcomes for both the Head-to-Head (H2H) and Free-For-All (F4A) experimentation phases using Optuna [4].

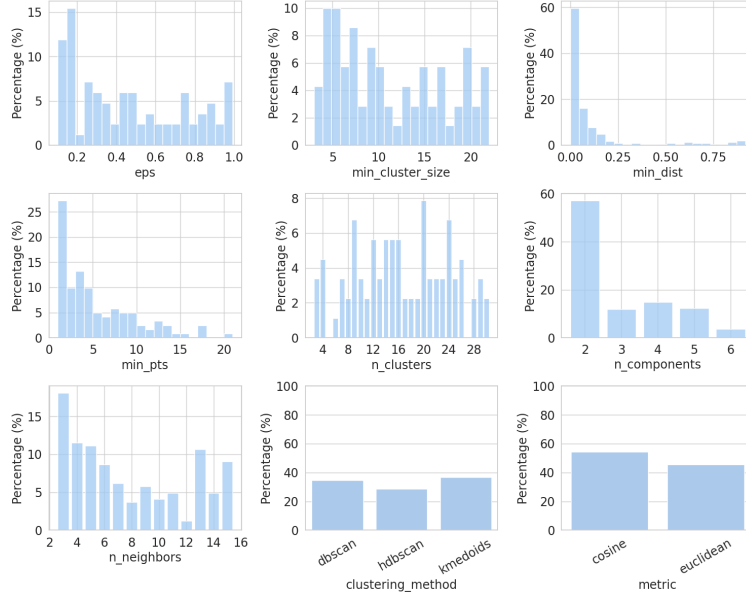
Section A.2.1 illustrates the distribution of hyperparameter values across the configurations identified as optimal by Optuna during the H2H and F4A runs, corresponding to Table 3.5. It presents these distributions first for all optimal configurations found across the various studies, and then specifically for the top 5 configurations selected via rank aggregation (as described in Section 3.6) for each study. These plots offer insights into which parameter settings were frequently favoured by the optimisation process across different embedding models (Overall, MPNet, Nomic).

Sections A.2.2 and A.2.4 display the results of the multi-objective optimisation studies. These 3-dimensional plots show the trade-offs between the Silhouette coefficient (higher is better), Calinski-Harabasz index (higher is better), and Davies-Bouldin index (lower is better) for the evaluated configurations. The best configuration found by Optuna, identified via rank aggregation (Section 3.6), are highlighted with red markers (top 5, or top 1 if fewer than 5 non-dominated solutions were found). Markers closer to the ideal top-left-front corner represent better solutions across the three heuristics; marker opacity indicates distance in the 3D space (more opaque is closer).

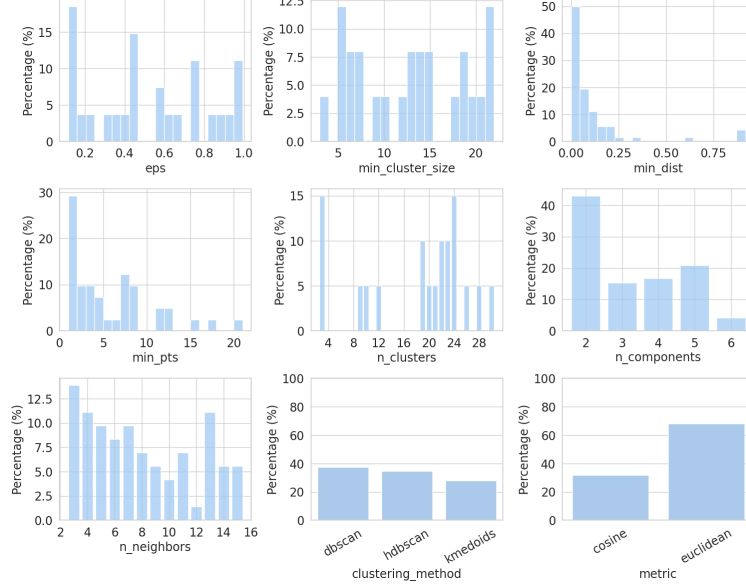
Sections A.2.3 and A.2.5 present whisker plots illustrating the distribution of cluster sizes obtained from the optimal configurations identified in the corresponding optimisation runs. These plots show the stability and typical scale of clusters produced by different pipeline settings over 10 independent runs (to account for UMAP stochasticity). Note that empty plots indicate that no competitive configurations using that specific clustering method were found among the optimal set for that particular Optuna study.

A.2.1 Aggregated Configuration Distributions

Overall



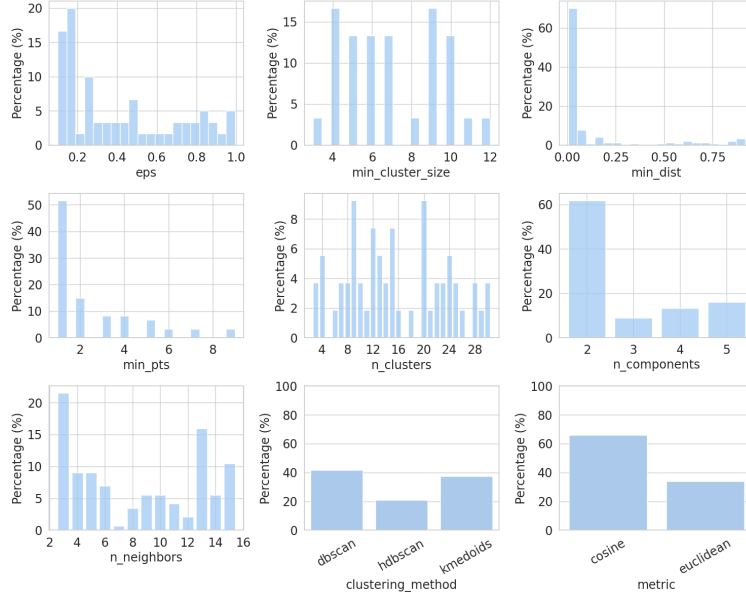
(a) All optimal configurations



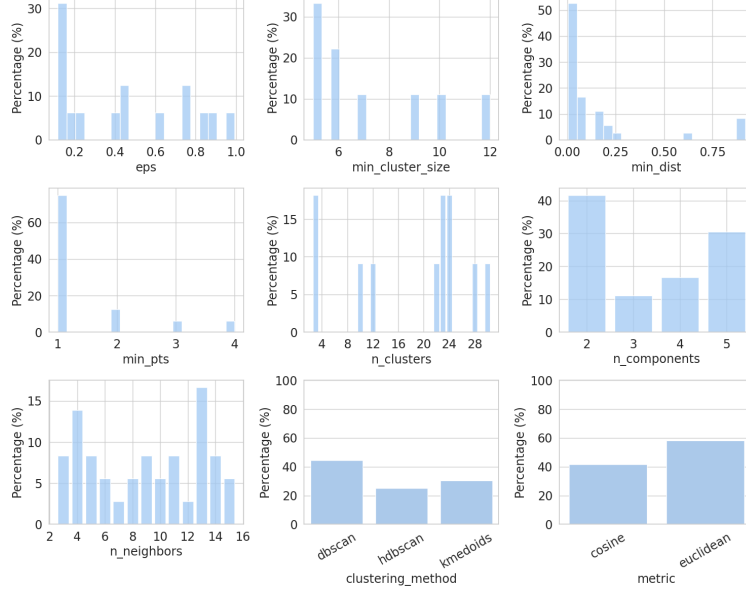
(b) Top 5 configurations (Rank Aggregated)

Figure A.7: Overall distribution of hyperparameter values across all optimal configurations identified by Optuna in H2H and F4A studies (top), and for the top 5 configurations selected via rank aggregation from each study (bottom).

MPNet



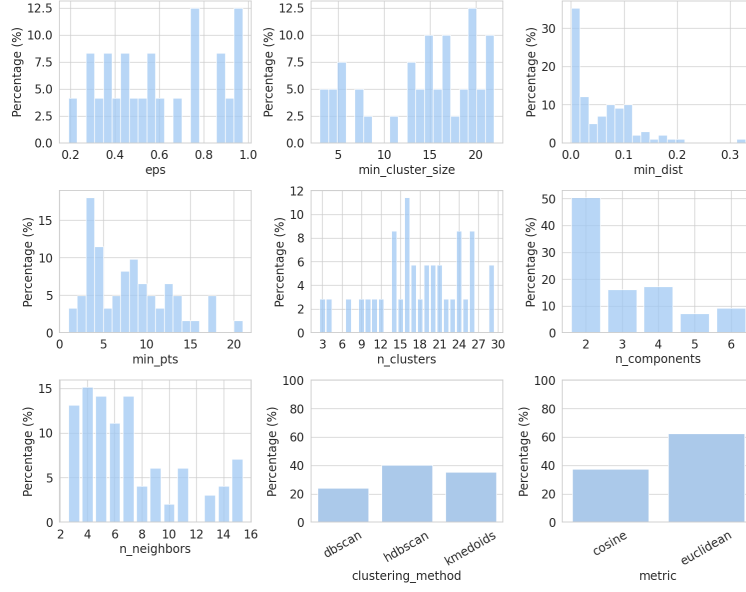
(a) All optimal configurations (MPNet)



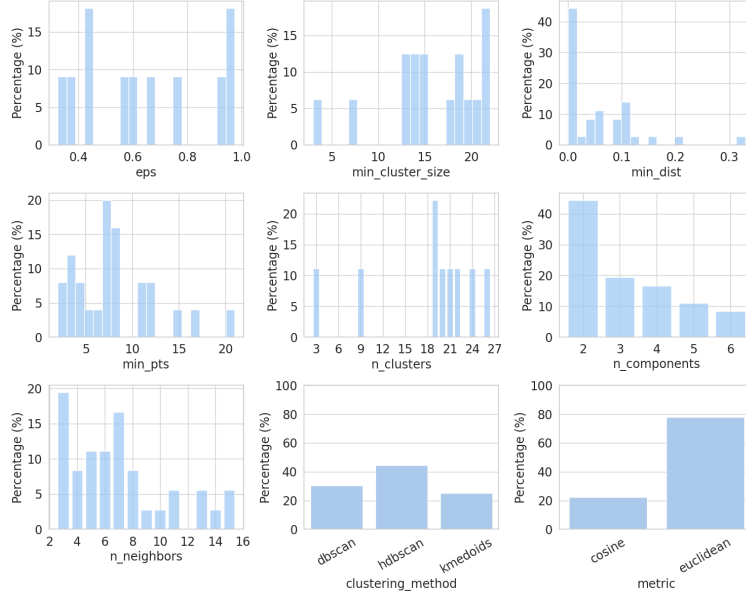
(b) Top 5 configurations (MPNet, Rank Aggregated)

Figure A.8: Distribution of hyperparameter values for MPNet embeddings across all optimal configurations identified by Optuna in H2H and F4A studies (top), and for the top 5 configurations selected via rank aggregation from each study (bottom).

Nomic



(a) All optimal configurations (Nomic)

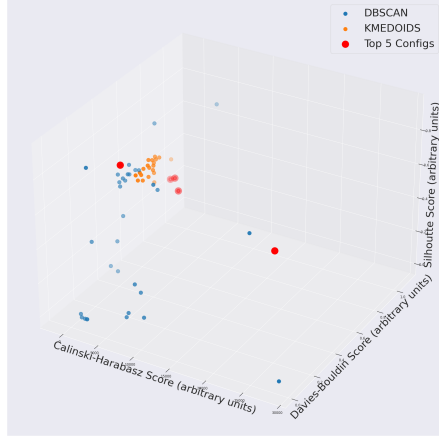


(b) Top 5 configurations (Nomic, Rank Aggregated)

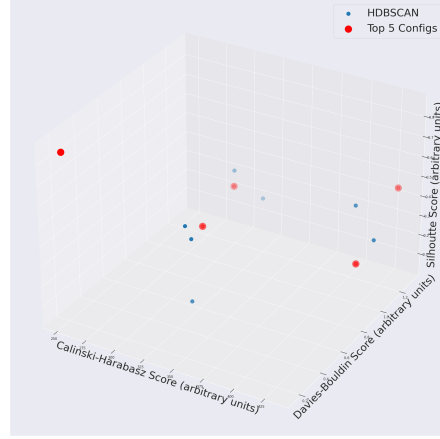
Figure A.9: Distribution of hyperparameter values for MPNet embeddings across all optimal configurations identified by Optuna in H2H and F4A studies (top), and for the top 5 configurations selected via rank aggregation from each study (bottom).

A.2.2 H2H Optimal Configuration Plots

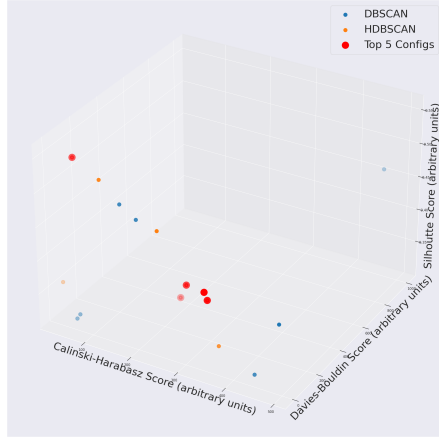
MPNet



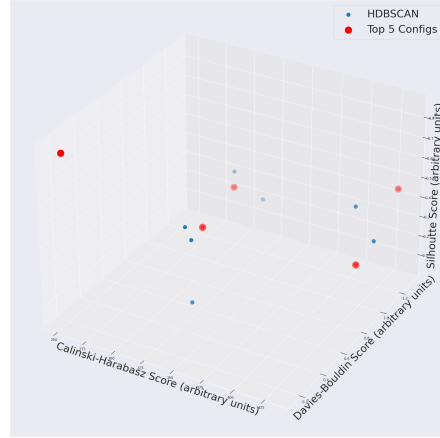
(a) Run A: k-Medoids vs. DBSCAN



(b) Run B: k-Medoids vs. HDBSCAN



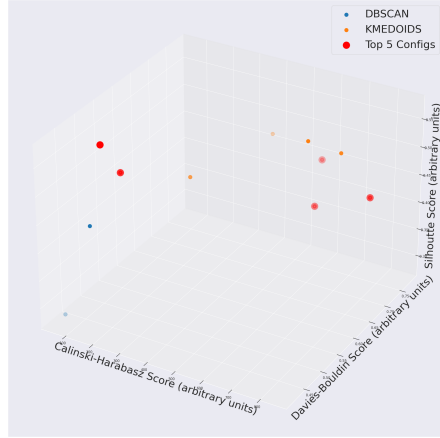
(c) Run C: DBSCAN vs. HDBSCAN



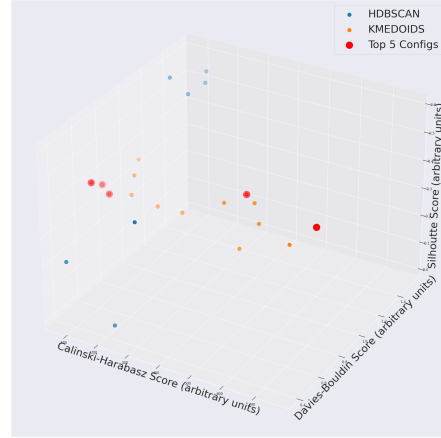
(d) Run D: HDBSCAN Only

Figure A.10: Heuristic trade-offs for optimal configurations from H2H Optuna runs using MPNet embeddings. Best solutions highlighted in red.

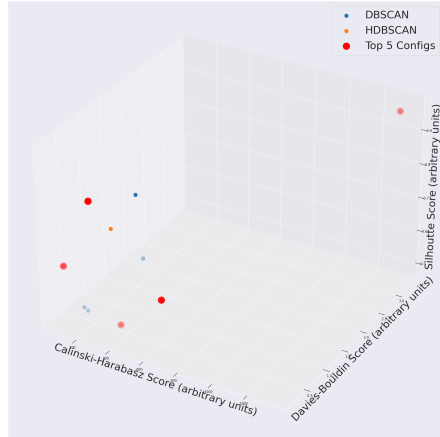
Nomic



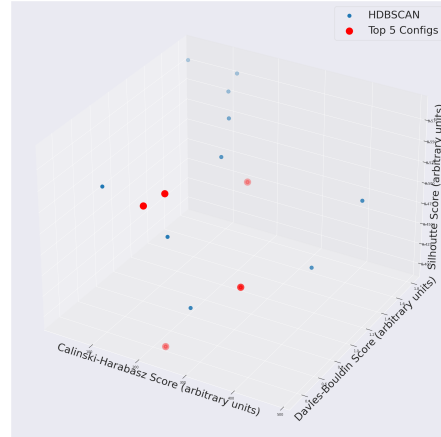
(a) Run A: k-Medoids vs. DBSCAN



(b) Run B: k-Medoids vs. HDBSCAN



(c) Run C: DBSCAN vs. HDBSCAN

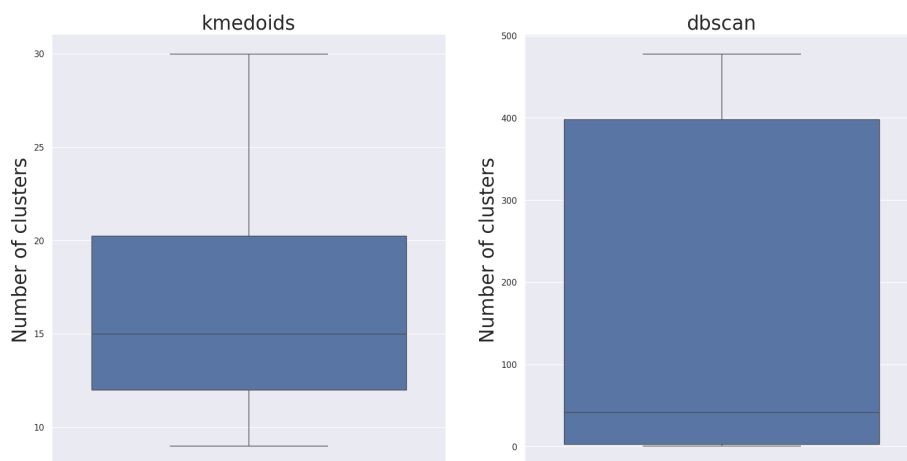


(d) Run D: HDBSCAN Only

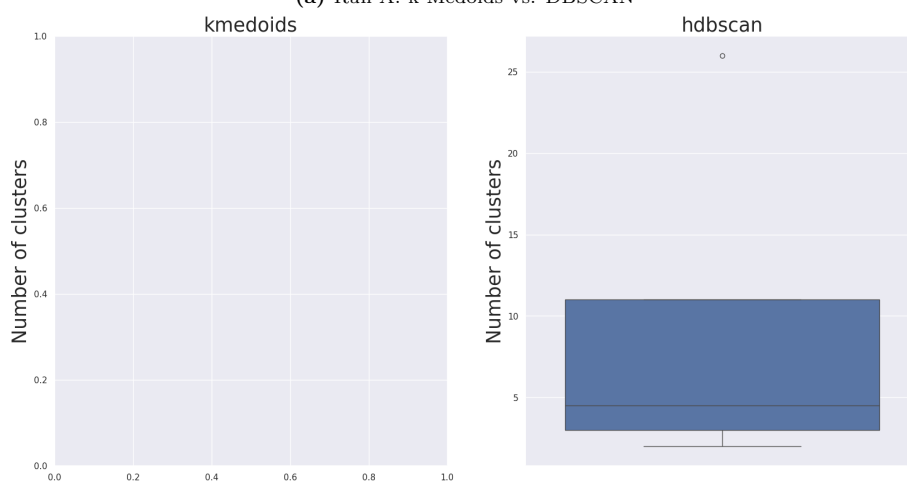
Figure A.11: Heuristic trade-offs for optimal configurations from H2H Optuna runs using Nomic embeddings. Best solutions highlighted in red.

A.2.3 H2H Cluster Size Distributions

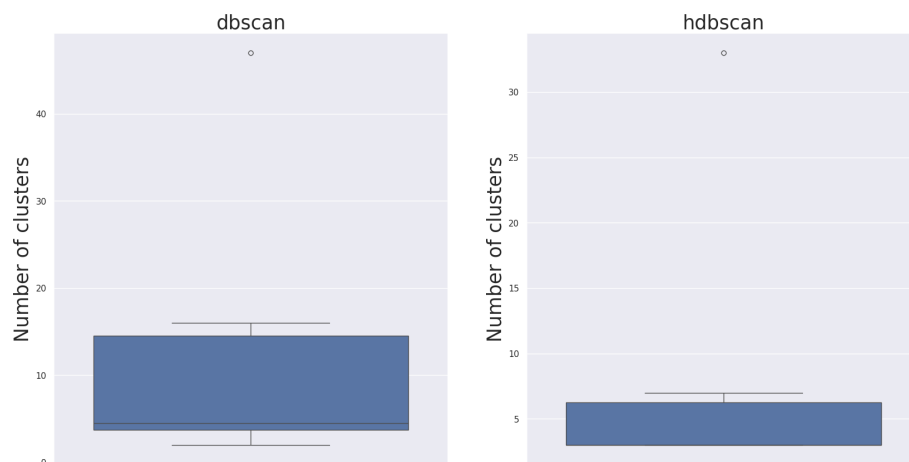
MPNet



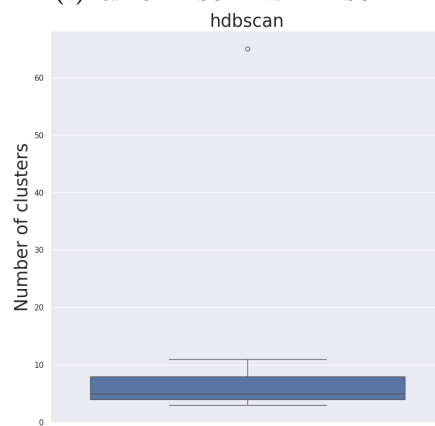
(a) Run A: k-Medoids vs. DBSCAN



(b) Run B: k-Medoids vs. HDBSCAN



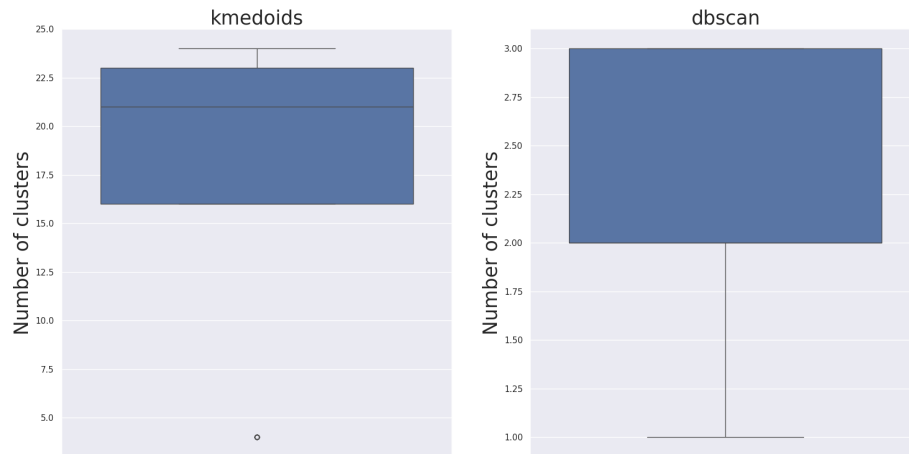
(c) Run C: DBSCAN vs. HDBSCAN



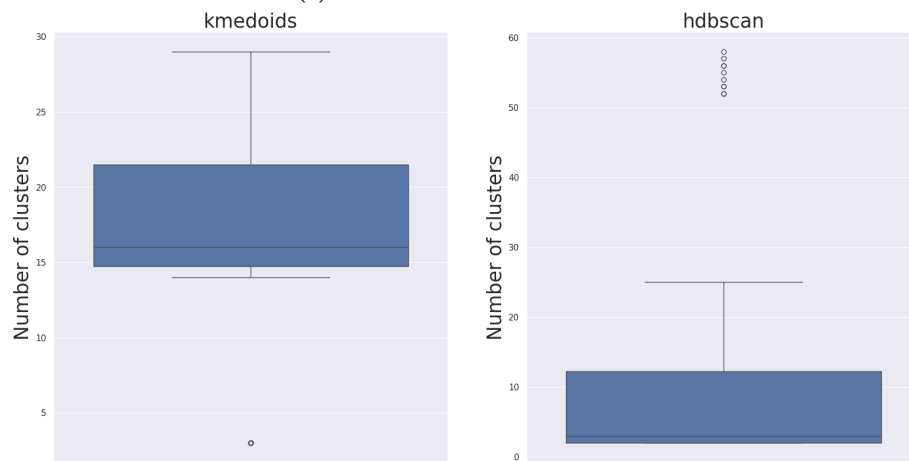
(d) Run D: HDBSCAN Only

Figure A.12: Cluster size distributions over 10 runs for optimal H2H configurations using MPNet embeddings. Empty plots indicate no optimal configurations were found for that method.

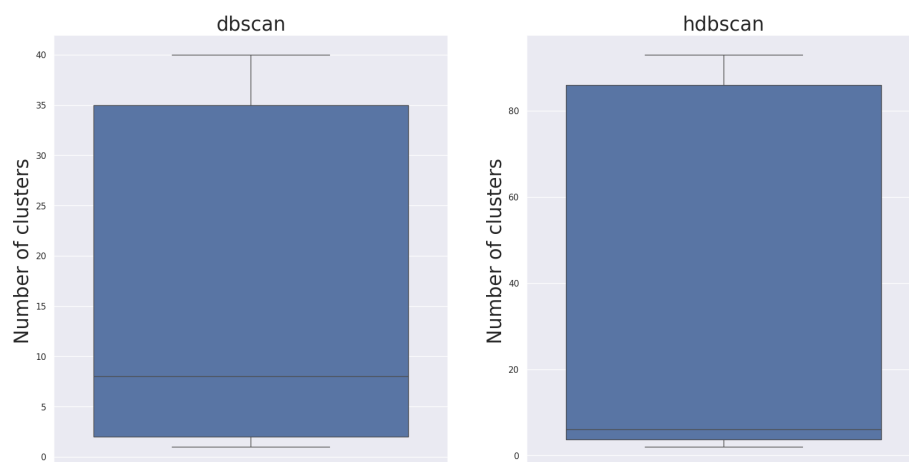
Nomic



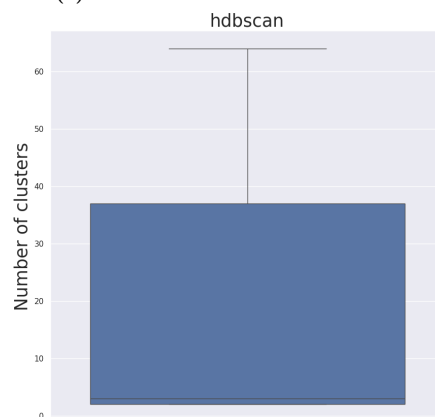
(a) Run A: k-Medoids vs. DBSCAN



(b) Run B: k-Medoids vs. HDBSCAN



(c) Run C: DBSCAN vs. HDBSCAN

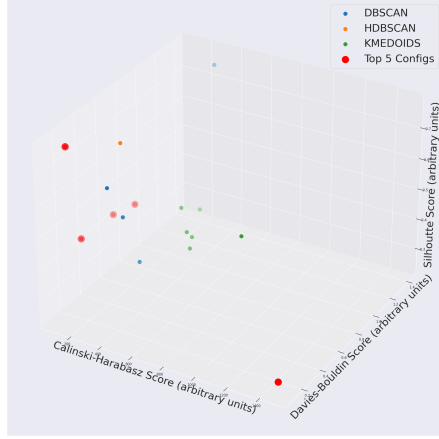


(d) Run D: HDBSCAN Only

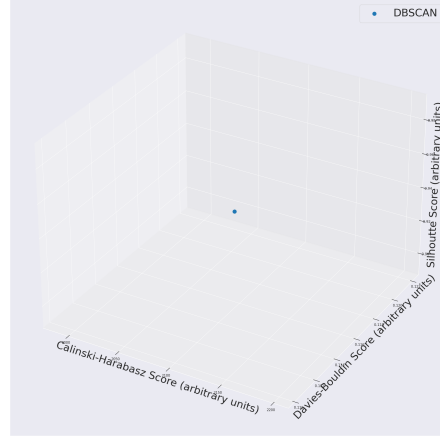
Figure A.13: Cluster size distributions over 10 runs for optimal H2H configurations using Nomic embeddings. Empty plots indicate no optimal configurations were found for that method.

A.2.4 F4A Optimal Configuration Plots

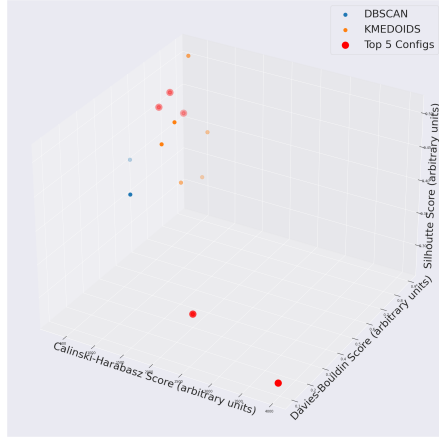
MPNet



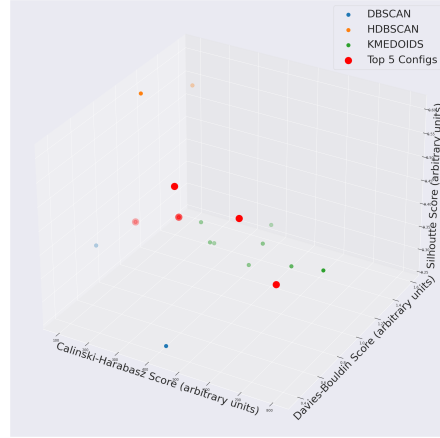
(a) ignore_case=False, keep_punct=False



(b) ignore_case=False, keep_punct=True



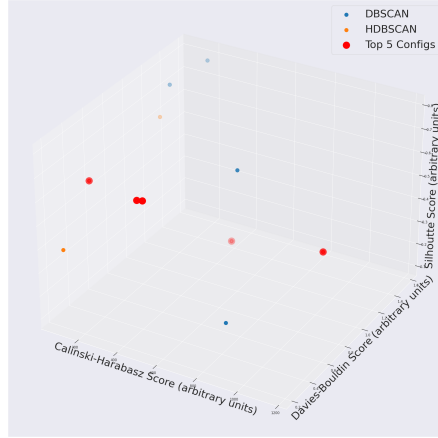
(c) ignore_case=True, keep_punct=False



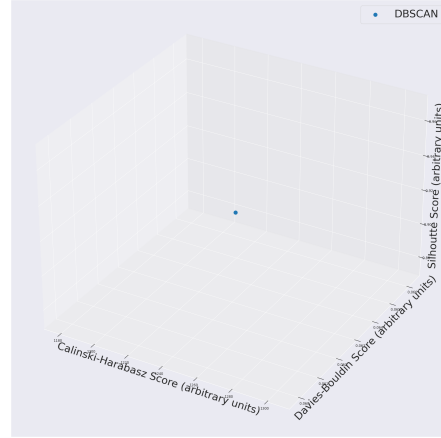
(d) ignore_case=True, keep_punct=True

Figure A.14: Heuristic trade-offs for optimal configurations from F4A Optuna runs using MPNet embeddings, varying pre-processing flags. Best solutions highlighted in red.

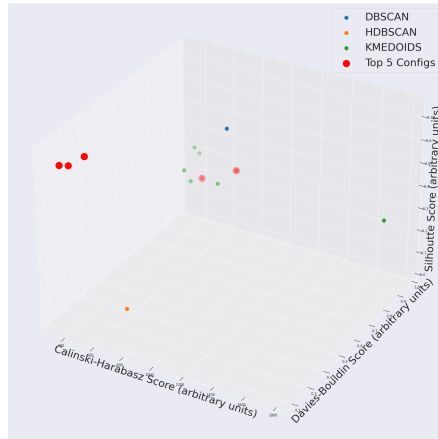
Nomic



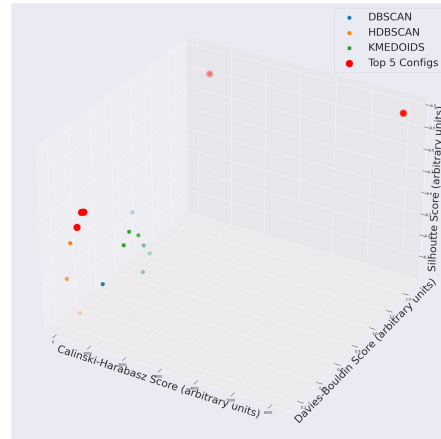
(a) ignore_case=False, keep_punct=False



(b) ignore_case=False, keep_punct=True



(c) ignore_case=True, keep_punct=False

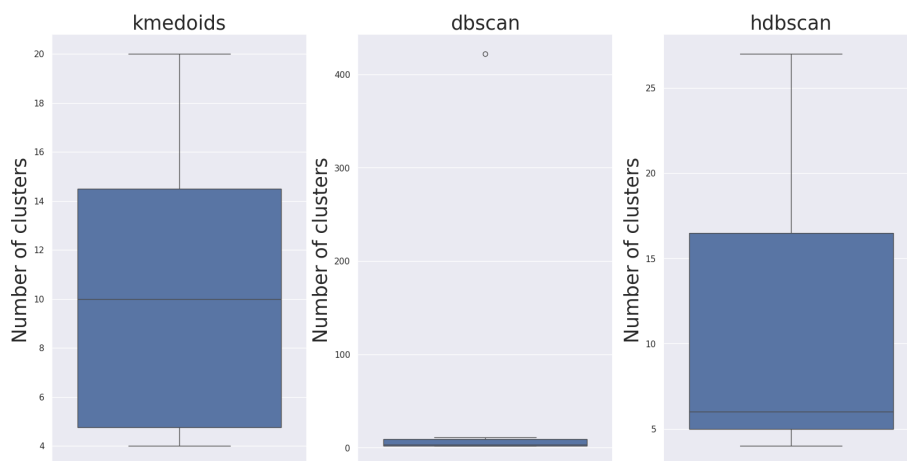


(d) ignore_case=True, keep_punct=True

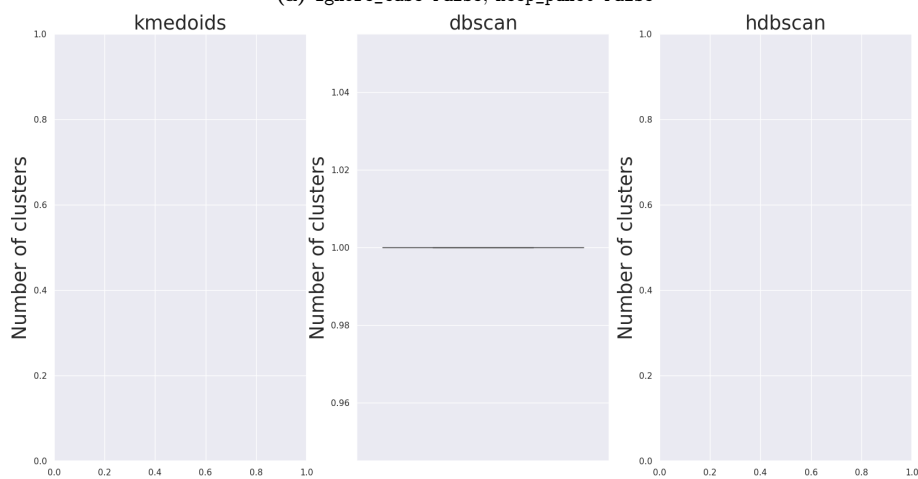
Figure A.15: Heuristic trade-offs for optimal configurations from F4A Optuna runs using Nomic embeddings, varying pre-processing flags. Best solutions highlighted in red.

A.2.5 F4A Cluster Size Distributions

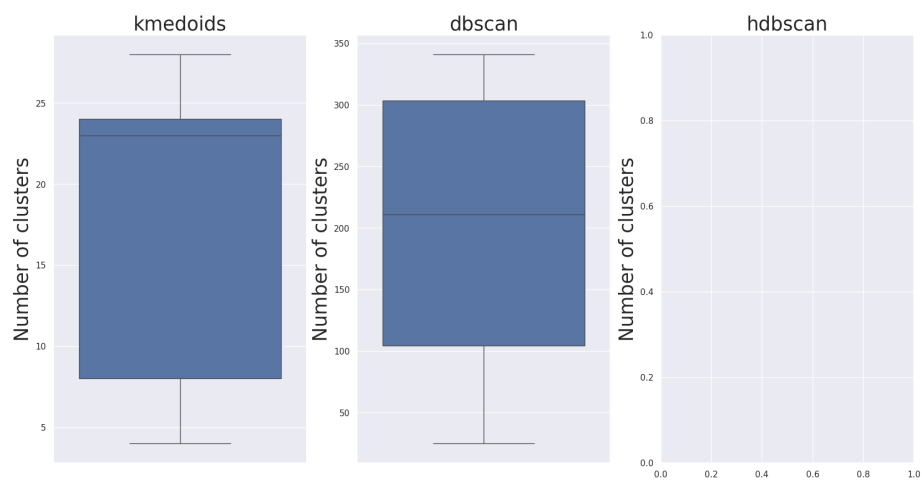
MPNet



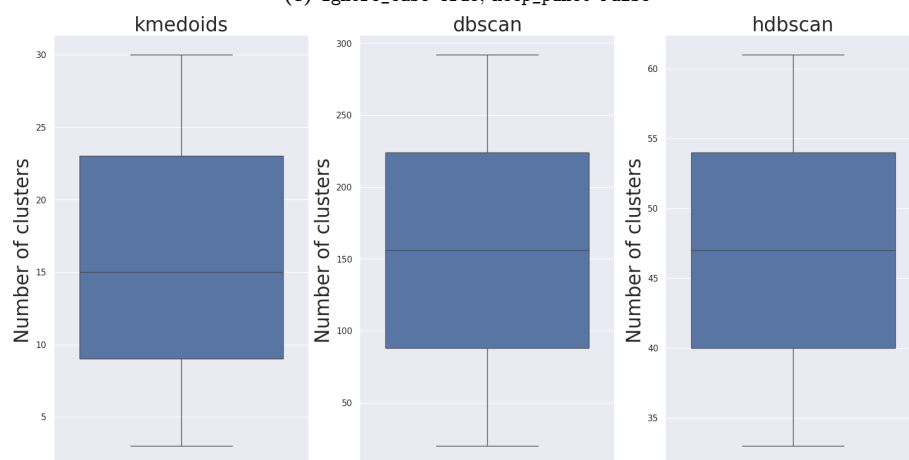
(a) ignore_case=False, keep_punct=False



(b) ignore_case=False, keep_punct=True



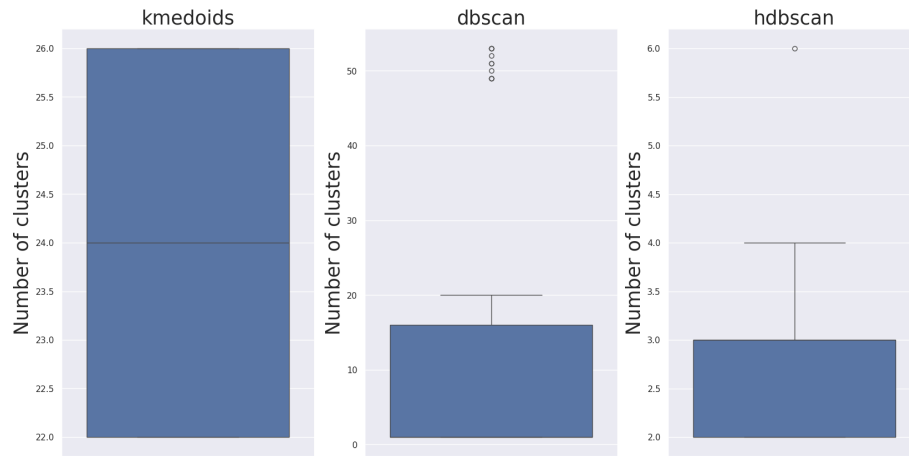
(c) `ignore_case=True, keep_punct=False`



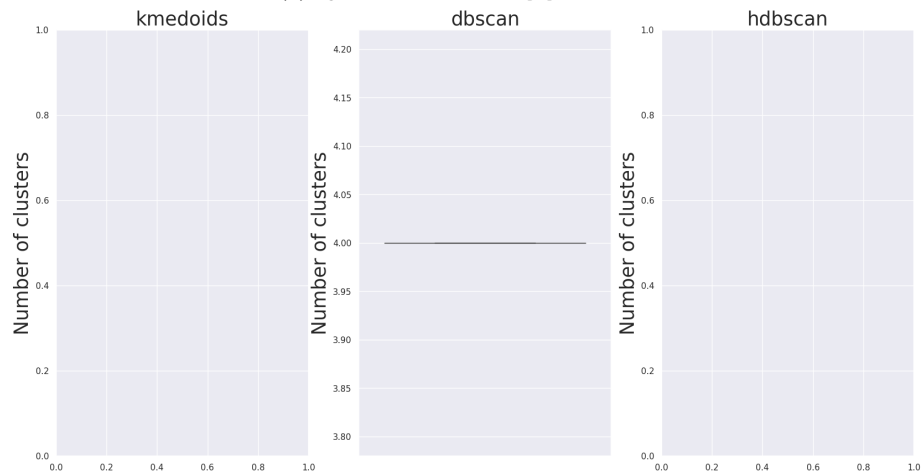
(d) `ignore_case=True, keep_punct=True`

Figure A.16: Cluster size distributions over 10 runs for optimal F4A configurations using MPNet embeddings, varying pre-processing flags.

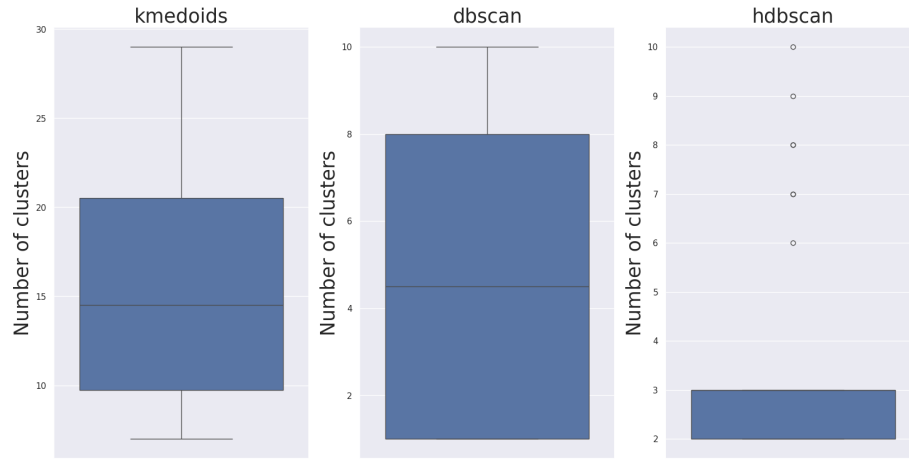
Nomic



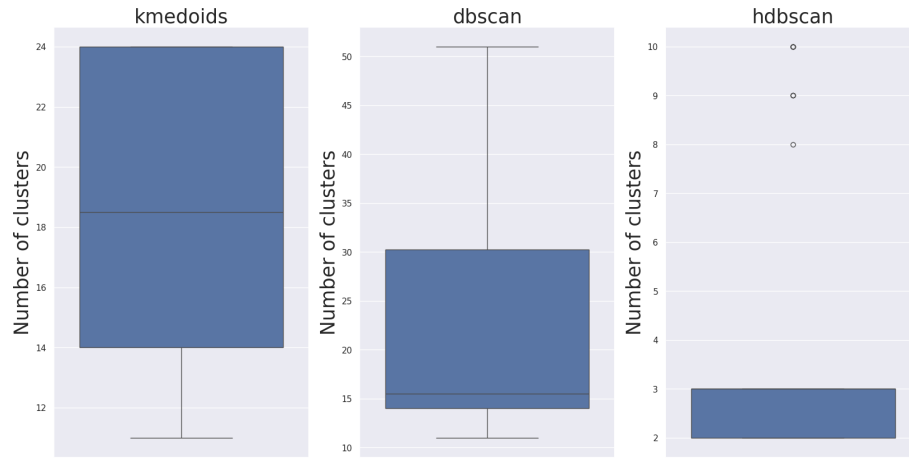
(a) ignore_case=False, keep_punct=False



(b) ignore_case=False, keep_punct=True



(c) ignore_case=True, keep_punct=False



(d) ignore_case=True, keep_punct=True

Figure A.17: Cluster size distributions over 10 runs for optimal F4A configurations using Nomic embeddings, varying pre-processing flags.

Appendix B

Supplementary Technical Details

B.1 BERT and XLNet Pre-training

This section provides supplementary technical details on the pre-training objectives and mechanisms of the BERT and XLNet models, as discussed conceptually in Section 2.2.1.

B.1.1 BERT Pre-training Tasks

The BERT model [24] employs two pre-training objectives during its pre-training phase:

- **Masked Language Modelling** - During training, a portion of the input tokens are ‘masked’ and the model predicts these masked tokens based on the remaining context, using cross-entropy loss [104]. In the literature, this is referred to as a Cloze task [87]. In the experiments conducted by (Devlin et al., 2019) 15% of tokens in each sequence are randomly selected for masking. To mitigate the mismatch between pre-training and fine-tuning phases where the [MASK] token does not appear, the training data generator replaces each chosen masked token with: (1) the special token [MASK] 80% of the time; (2) a random token 10% of the time; and (3) the original token itself 10% of the time [24].
- **Next Sentence Prediction** - In this task, the model is trained on a binary classification task predicting the relationship between sentence pairs. Given two sentences A and B either (1) sentence B is the actual sentence that follows A 50% of the time or (2) sentence B is randomly selected from the corpus 50% of the time [24]. This task aimed to help BERT understand sentence relationships, though later models like RoBERTa and MPNet found it might not always be beneficial [50, 79].

Figure B.1 illustrates the overall pre-training and fine-tuning workflow for BERT.

B.1.2 XLNet Technical Details

XLNet [101] introduced mechanisms to overcome limitations observed in BERT, particularly concerning the pre-train-to-fine-tune discrepancy and the independence assumption in MLM.

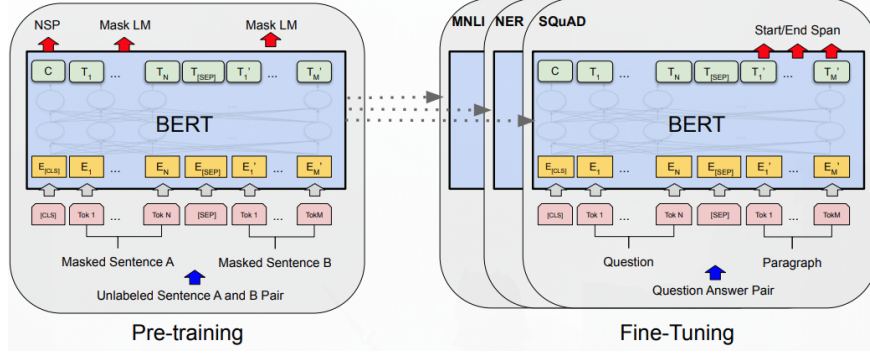


Figure B.1: The overall pre-training and fine-tuning phases for BERT. For different downstream tasks, notice the same architecture, except output layer, is used. Source: (Devlin et al., 2019). (Figure originally appeared as Figure 2.2)

Auto-regressive vs. Auto-encoding Perspectives

According to (Yang et al., 2019), pre-training objectives can be broadly categorized as auto-regressive (AR) or auto-encoding (AE). AR language modelling, given a sequence of tokens $X = (x_1, \dots, x_T)$, aims to factorize the likelihood of a token into either a forward or backward product:

$$p(x) = \prod_{i=1}^T p(x_i | X_{<i}) \quad (\text{B.1})$$

$$p(x) = \prod_{i=T}^1 p(x_i | X_{>i}) \quad (\text{B.2})$$

where $X_{<i}$ denotes tokens before position i . As standard AR models only encode context unidirectionally, their effectiveness can be limited [101]. AE models like BERT reconstruct original data from corrupted input (i.e. masked tokens), allowing bidirectional context but potentially suffering from issues like the pretrain-finetune discrepancy and independence assumptions [101].

Permutation Language Modeling Formalism

XLNet employs Permutation Language Modelling as an AR objective that captures bidirectional context. It maximizes the expected log-likelihood over all possible factorisation orders (permutations) of the input sequence [101]. More formally:

$$\max_{\theta} \mathbb{E}_{z \in \mathcal{Z}_T} \left[\sum_{i=1}^T \log p_{\theta}(x_{z_i} | x_{z_{<i}}) \right], \quad (\text{B.3})$$

where $\mathcal{Z}_T$ is the set of all $T!$ permutations of the index list $[1, \dots, T]$. For a given permutation $z = (z_1, \dots, z_T)$, $X_{z_{<i}}$ denotes the tokens appearing before z_i in that permutation. θ

represents the shared model parameters across all permutations [101]. By training across all permutations, each position learns to utilise context from all other positions.

Two-Stream Self-Attention Mechanism

To implement permutation language modelling while preventing the model from seeing the token it needs to predict, XLNet uses a two-stream self-attention mechanism [91, 101]. It maintains two hidden states per position at each layer:

1. **Content Stream** (h) - Encodes both the content and position of a token, similar to standard transformer hidden states. It can access contextual information up to and including the current position in the permutation.
2. **Query Stream** (g) - Encodes contextual information and the current position z_i , but, importantly, not the content x_{z_i} of the token at the current position. It is used to gather information from the context $x_{z < i}$ to predict x_{z_i} .

At layer m and target position z_i , the streams are updated using attention mechanisms, conceptually as follows [91, 101]:

$$g_{z_i}^{(m)} \leftarrow \text{Attention}(Q = g_{z_i}^{(m-1)}, KV = h_{z < i}^{(m-1)}) \quad (\text{B.4})$$

$$h_{z_i}^{(m)} \leftarrow \text{Attention}(Q = h_{z_i}^{(m-1)}, KV = h_{z \leq i}^{(m-1)}) \quad (\text{B.5})$$

The query stream $g_{z_i}^{(m)}$ attends only to the content representations h of tokens preceding z_i in the current permutation. The content stream $h_{z_i}^{(m)}$ calculates an attention distribution over tokens up to and including z_i . After the final layer M , the query stream representation $g_{(z_i)}^{(M)}$ is used to compute the probability distribution over the vocabulary for predicting x_{z_i} via a soft-max layer [28, 101]. This mechanism allows XLNet to predict tokens using contextual information gathered auto-regressively based on the permutation, without directly accessing the target token’s content representation.

B.2 MPNet Mathematical Formulations

This section provides supplementary mathematical details for the MPNet model discussed in Section 3.3.2.

B.2.1 MPNet Pre-training Objective

The pre-training objective for MPNet aims to maximize the expected log-likelihood of predicting tokens based on a permuted context that includes information about masked tokens later in the sequence. Formally, let $\mathcal{Z}_T$ be the set of all $T!$ permutations of the index list $[1, \dots, T]$. For a given permutation $z = (z_1, \dots, z_T)$, let x_{z_i} denote the token at original position z_i , and $x_{z < i}$ denote the sequence of tokens preceding z_i in the permutation z . Let M be the set of indices corresponding to masked tokens, and $M_{z > c}$ be the subset of masked token indices appearing after position c in the permutation z . The objective is defined as [79]:

$$\max_{\theta} \mathbb{E}_{z \in \mathcal{Z}_T} \left[\sum_{i=1}^T \log p_{\theta}(x_{z_i} \mid x_{z < i}, M_{z > c}) \right] \quad (\text{B.6})$$

where θ represents the shared model parameters, fixed for each permutation z .

B.2.2 MPNet Input Structuring Example

To implement the objective (Equation B.6), the input sequence is rearranged based on the permutation z and a split point c . The sequence is divided into non-predicted tokens ($x_{z \leq c}$) and predicted tokens ($x_{z > c}$). The input representation fed to the model concatenates the non-predicted part, mask token placeholders corresponding to the predicted part ($M_{z > c}$), and the actual predicted tokens (used only for loss calculation). For an example input sequence $(x_1, \dots, x_6)$, a permutation $z = (x_3, x_5, x_2, x_1, x_4, x_6)$, and a split point $c = 3$, the structured input is represented as [79]:

$$\langle x_{z \leq c}; M_{z > c}; x_{z > c} \rangle = \langle x_3, x_5, x_2; [M], [M], [M]; x_1, x_4, x_6 \rangle \quad (\text{B.7})$$

where $[M]$ denotes the mask token placeholder.

B.2.3 PCA Formalism and SVD Derivation

This appendix provides the formal mathematical details underpinning the conceptual description of Principal Component Analysis (PCA) presented in Section 2.3.1, including its optimisation objective and connection to Singular Value Decomposition (SVD) [81].

If we want to reduce the dimensionality from n features down to k principal components (for $k < n$), we seek a matrix $C \in \mathbb{R}^{n \times k}$ whose columns represent these top k orthogonal component directions (i.e. columns are statistically uncorrelated). The projected data in the lower k -dimensional space is given by XC . We can approximate the original data X by projecting back from this lower dimension using $XC C^T$. PCA aims to find the matrix C that minimises the difference between the original data X and this reconstructed data $XC C^T$, subject to the constraint that the column-wise components are orthogonal ($C^T C = I$, for the identity matrix I) [89]:

$$\min_C \|X - XC C^T\|_F^2 \quad (\text{B.8})$$

A standard and efficient method for solving this optimisation problem and finding the principal components is through the Singular Value Decomposition (SVD) of the data matrix X [81, 89]. SVD is a fundamental matrix factorisation technique that decomposes X into three matrices:

$$X = USV^T \quad (\text{B.9})$$

Here, $U \in \mathbb{R}^{m \times k}$ and $V \in \mathbb{R}^{n \times k}$ are matrices with orthogonal columns, $S \in \mathbb{R}^{k \times k}$ is a diag-

onal matrix (only the diagonal contains non-zero elements) which contains the non-negative singular values (s_i) sorted in descending order, and k is the rank of matrix X .

The crucial link to PCA is that the columns of the matrix V are precisely the principal component directions we seek. That is, the matrix C in Equation B.8, containing the top k principal components, corresponds to the first k columns of V .

Furthermore, SVD connects directly to the covariance matrix of the data ($X^T X$, assuming X is centered). The decomposition of the covariance matrix is given by:

$$X^T X = (USV^T)^T (USV^T) \quad (\text{B.10})$$

$$= (VS^T U^T)(USV^T) \quad (\text{B.11})$$

$$= VS^T (U^T U) SV^T \quad (\text{B.12})$$

$$= VS^T I SV^T \quad (\text{since } U^T U = I) \quad (\text{B.13})$$

$$= V(S^T S)V^T \quad (\text{B.14})$$

$$= VS^2 V^T \quad (\text{since } S \text{ is a diagonal matrix}) \quad (\text{B.15})$$

This is the eigen-decomposition of the covariance matrix $X^T X$. It shows that the columns of V are the eigenvectors (the principal components), and the diagonal entries of S^2 are the corresponding eigenvalues (i.e. $s_i^2 = \lambda_i$). Since the eigenvalues represent the variance of the data along the eigenvectors, the squared singular values (s_i^2) directly quantify the variance captured by each principal component [89]. Therefore, computing the SVD of the data matrix X provides both the principal components (in V) and the measure of variance associated with each component (in S^2) without explicitly forming the covariance matrix. Using a truncated SVD (keeping only the top k components corresponding to the largest singular values) gives us a reduction in the dimensions.

B.3 k-Medoids - The PAM Algorithm

This section provides a high-level pseudocode of the Partitioning Around Medoids (PAM) algorithm, the core algorithm of the k-Medoids clustering method, as discussed in Section 2.4.1 [38].

Algorithm B.1: Partitioning Around Medoids (PAM).

```

1  input: Dataset  $X = \{x_1, \dots, x_n\}$ , integer  $k$  (number of clusters)
2  output: Set of  $k$  medoids  $M$ , Partition  $X' = \{C_1, \dots, C_k\}$  where  $C_j$  is the set of points
        assigned to medoid  $m_j$ 
3
4  begin
5      # --- Build Phase (Conceptual) ---
6      # Select an initial set of  $k$  medoids  $M$  from  $X$ 
7      # (i.e. using a heuristic that minimises initial total dissimilarity)
8       $M \leftarrow \text{SelectInitialMedoids}(X, k)$ 
9
10     # --- Swap Phase ---
11     repeat
12         best_cost_reduction  $\leftarrow 0$ 
13         best_swap  $\leftarrow \emptyset$ 
14         total_cost  $\leftarrow \text{ComputeTotalDissimilarity}(X, M)$  # Cost based on current  $M$ 
15
16         # Consider swapping each medoid  $m$  with each non-medoid  $x$ 
17         foreach  $m \in M$  do
18             foreach  $x \in X \setminus M$  do
19                 # Compute cost if  $m$  is replaced by  $x$ 
20                  $M_{\text{swap}} \leftarrow (M \setminus \{m\}) \cup \{x\}$ 
21                 cost_after_swap  $\leftarrow \text{ComputeTotalDissimilarity}(X, M_{\text{swap}})$ 
22                 cost_reduction  $\leftarrow \text{total\_cost} - \text{cost\_after\_swap}$ 
23
24                 # Check if this swap is the best found so far
25                 if cost_reduction > best_cost_reduction then
26                     best_cost_reduction  $\leftarrow \text{cost\_reduction}$ 
27                     best_swap  $\leftarrow (m, x)$  # Store the potential swap pair
28                 end
29             end
30         end
31
32         # Perform the best swap found in this iteration, if any cost reduction
33         # occurred
34         if best_cost_reduction > 0 then
35              $(m_{\text{out}}, x_{\text{in}}) \leftarrow \text{best\_swap}$ 
36              $M \leftarrow (M \setminus \{m_{\text{out}}\}) \cup \{x_{\text{in}}\}$  # Update medoid set
37             improved  $\leftarrow \text{true}$ 
38         else
39             improved  $\leftarrow \text{false}$  # No cost reduction possible
40         end
41     until not improved # Stop when no swap improves the total cost
42
43     # Assign points to final medoids to form clusters  $X'$ 
44      $X' \leftarrow \text{AssignPointsToClusters}(X, M)$ 
45
46     return  $M, X'$ 
47 end

```
